

Clases



... y Objetos



Classes

```
class Bicycle {  
  
    // three fields  
    int cadence;  
    int gear;  
    int speed;  
  
    // the Bicycle class has  
    // one constructor  
    Bicycle(int startCadence,  
            int startSpeed, int startGear) {  
        gear = startGear;  
        cadence = startCadence;  
        speed = startSpeed;  
    }  
  
    // four methods  
    void setCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void setGear(int newValue) {  
        gear = newValue;  
    }  
  
    void applyBrake(int decrement) {  
        speed -= decrement;  
    }  
  
    void speedUp(int increment) {  
        speed += increment;  
    }  
}
```

```
class MountainBike extends Bicycle {  
  
    // one field  
    int seatHeight;  
  
    // the MountainBike subclass has  
    // one constructor  
    MountainBike(int startHeight,  
                 int startCadence,  
                 int startSpeed,  
                 int startGear) {  
        super(startCadence, startSpeed,  
              startGear);  
        seatHeight = startHeight;  
    }  
  
    // one method  
    void setHeight(int newValue) {  
        seatHeight = newValue;  
    }  
}
```


Classes

```
class Bicycle {  
  
    // three fields  
    int cadence;  
    int gear;  
    int speed;  
  
    // the Bicycle class has  
    // one constructor  
    Bicycle(int startCadence,  
            int startSpeed, int startGear) {  
        gear = startGear;  
        cadence = startCadence;  
        speed = startSpeed;  
    }  
  
    // four methods  
    void setCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void setGear(int newValue) {  
        gear = newValue;  
    }  
  
    void applyBrake(int decrement) {  
        speed -= decrement;  
    }  
  
    void speedUp(int increment) {  
        speed += increment;  
    }  
}
```

```
class MountainBike extends Bicycle {  
  
    // one field  
    int seatHeight;  
  
    // the MountainBike subclass has  
    // one constructor  
    MountainBike(int startHeight,  
                  int startCadence,  
                  int startSpeed,  
                  int startGear) {  
        super(startCadence, startSpeed,  
               startGear);  
        seatHeight = startHeight;  
    }  
  
    // one method  
    void setHeight(int newValue) {  
        seatHeight = newValue;  
    }  
}
```


Classes

```
class Bicycle {  
  
    // three fields  
    int cadence;  
    int gear;  
    int speed;  
  
    // the Bicycle class has  
    // one constructor  
    Bicycle(int startCadence,  
            int startSpeed, int startGear) {  
        gear = startGear;  
        cadence = startCadence;  
        speed = startSpeed;  
    }  
  
    // four methods  
    void setCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void setGear(int newValue) {  
        gear = newValue;  
    }  
  
    void applyBrake(int decrement) {  
        speed -= decrement;  
    }  
  
    void speedUp(int increment) {  
        speed += increment;  
    }  
}
```

```
class MountainBike extends Bicycle {  
  
    // one field  
    int seatHeight;  
  
    // the MountainBike subclass has  
    // one constructor  
    public MountainBike(int startHeight,  
                        int startCadence,  
                        int startSpeed,  
                        int startGear) {  
        super(startCadence, startSpeed,  
              startGear);  
        seatHeight = startHeight;  
    }  
  
    // one method  
    void setHeight(int newValue) {  
        seatHeight = newValue;  
    }  
}
```


Clases

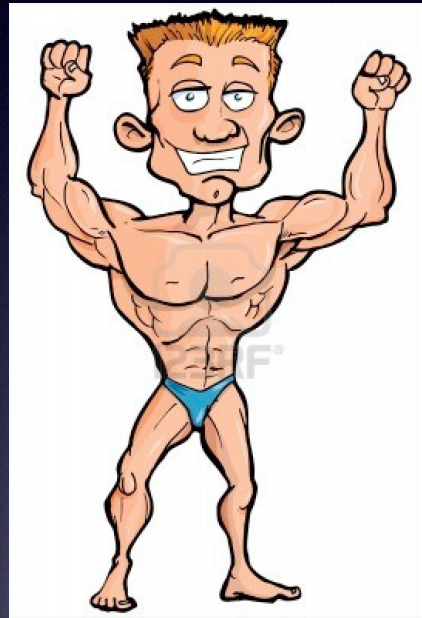
Declaración de clase:

```
class MyClass {  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
}
```


Clases

Declaración de clase:

```
class MyClass {
```



constructores, y
declaraciones

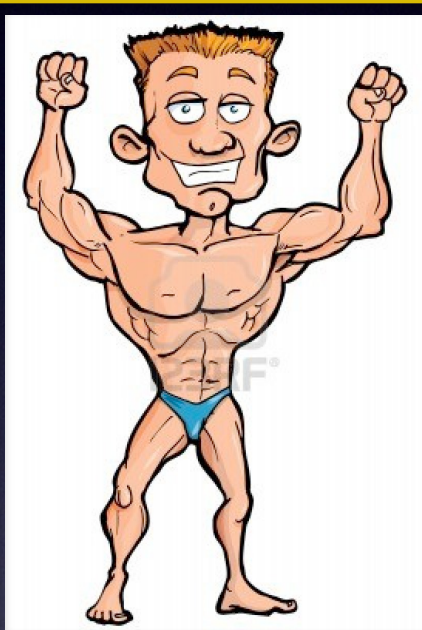
```
}
```

El cuerpo de la clase

Clases

Declaración de clase:

```
class MyClass extends MySuperClass implements YourInterface {
```



constructores, y
declaraciones

```
}
```


Classes

Declaración de clase:

```
public  } class MyClass extends MySuperClass  
private } implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Classes

Declaración de clase:

```
public  
private } class MyClass extends MySuperClass  
                                implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Classes

Declaración de clase:

```
public } class MyClass extends MySuperClass  
private } implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Classes

Declaración de clase:

```
public  } class MyClass extends MySuperClass  
private } implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Classes

Declaración de clase:

```
public  } class MyClass extends MySuperClass  
private } implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Classes

Declaración de clase:

```
public } class MyClass extends MySuperClass  
private } implements YourInterface {  
  
    ...  
    // atributos, constructores, y  
    // declaración de métodos  
    ...  
  
}
```


Tipos de variables

- Atributos. (Variables de instancia) (Fields en inglés)
- Atributos estáticos. (Variables de clase)
- Variables locales
- Parámetros

Tipos de

- Atributos. (Variables)
- Atributos estáticos.
- Variables locales
- Parámetros

```
class Bicycle {
    int cadence = 0;
    int speed = 0;
    int gear = 1;
    static int numBikes = 0;
    static final int MAX_GEARs = 8;

    void changeCadence(int newValue) {
        cadence = newValue;
    }

    void changeGear(int newValue) {
        int i = 0;
        ...
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
}
```


Tipos de Atributos

- Atributos. (Variables)
- Atributos estáticos.
- Variables locales
- Parámetros

```
class Bicycle {  
    int cadence = 0;  
    int speed = 0;  
    int gear = 1;  
    static int numBikes = 0;  
    static final int MAX_GEAR = 8;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        int i = 0;  
        ...  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```


Tipos de

Atributos estáticos

MAX_GEARs es una constante

- Atributos. (Variables)
- Atributos estáticos.
- Variables locales
- Parámetros

```
class Bicycle {  
    int cadence = 0;  
    int speed = 0;  
    int gear = 1;  
    static int numBikes = 0;  
    static final int MAX_GEARs = 8;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        int i = 0;  
        ...  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```


Tipos de

Variable local

- Atributos. (Variables)
- Atributos estáticos.
- Variables locales
- Parámetros

```
class Bicycle {
    int cadence = 0;
    int speed = 0;
    int gear = 1;
    static int numBikes = 0;
    static final int MAX_GEARs = 8;

    void changeCadence(int newValue) {
        cadence = newValue;
    }

    void changeGear(int newValue) {
        int i = 0;
        ...
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
}
```


Tipos de

Parámetros

- Atributos. (Variables)
- Atributos estáticos.
- Variables locales
- Parámetros

```
class Bicycle {
    int cadence = 0;
    int speed = 0;
    int gear = 1;
    static int numBikes = 0;
    static final int MAX_GEARs = 8;

    void changeCadence(int newValue) {
        cadence = newValue;
    }

    void changeGear(int newValue) {
        int i = 0;
        ...
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
}
```


Modificadores de acceso para los atributos

- Public
- Private
- ...

```
class Bicycle {  
    int cadence = 0;  
    int speed = 0;  
    int gear = 1;  
    static int numBikes = 0;  
    static final int MAX_GEARs = 8;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        int i = 0;  
        ...  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```


Modificadores de acceso para los atributos

- Public
- Private
- ...

```
class Bicycle {  
    private int cadence = 0;  
    private int speed = 0;  
    private int gear = 1;  
    static int numBikes = 0;  
    private static final int MAX_GEARs = 8;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        int i = 0;  
        ...  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```


Modificadores de acceso para los atributos y métodos

- Public
- Private
- ...

```
class Bicycle {  
    private int cadence = 0;  
    private int speed = 0;  
    private int gear = 1;  
    static int numBikes = 0;  
    private static final int MAX_GEARs = 8;  
  
    public void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    public void changeGear(int newValue) {  
        int i = 0;  
        ...  
        gear = newValue;  
    }  
  
    public void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    public void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    public void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```


Declaración de métodos

Ejemplo

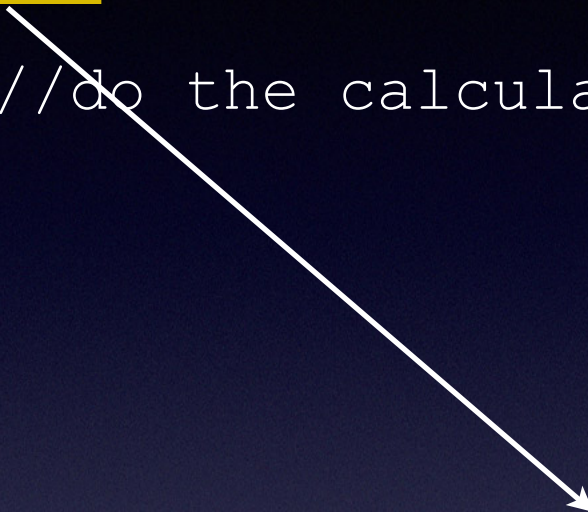
```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
                             double length, double grossTons) {  
    //do the calculation here  
}
```


Declaración de métodos

Ejemplo

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
                             double length, double grossTons) {  
    //do the calculation here  
}
```

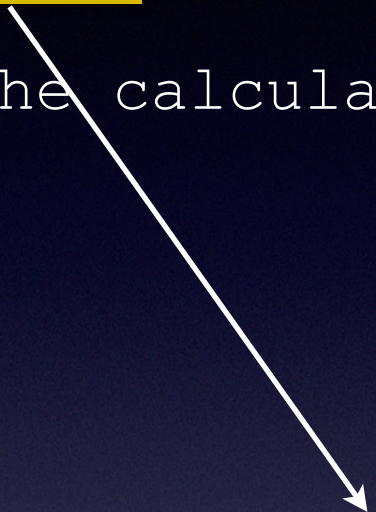
Modificadores {
 public
 private
 ...



Declaración de métodos

Ejemplo

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
                                double length, double grossTons) {  
    //do the calculation here  
}
```



Tipo que devuelve

void = nada

Declaración de métodos

Ejemplo

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
    //do the calculation here  
    double length, double grossTons) {  
}
```



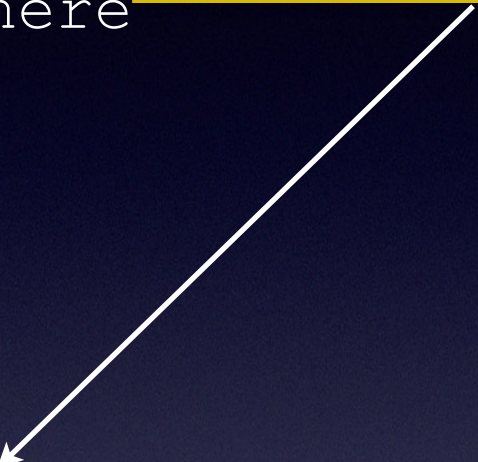
nombre

Declaración de métodos

Ejemplo

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
    double length, double grossTons) {  
    //do the calculation here  
}
```

Lista de parámetros

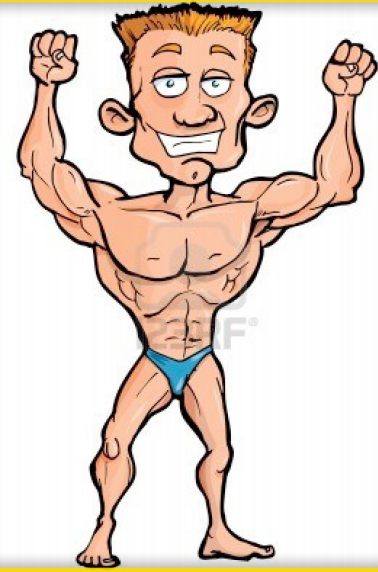


Declaración de métodos

Ejemplo

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
                             double length, double grossTons) {
```

```
//do the calculation here
```

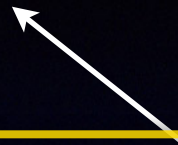


El cuerpo del método

Declaración de métodos

Ejemplo

Lista de parámetros



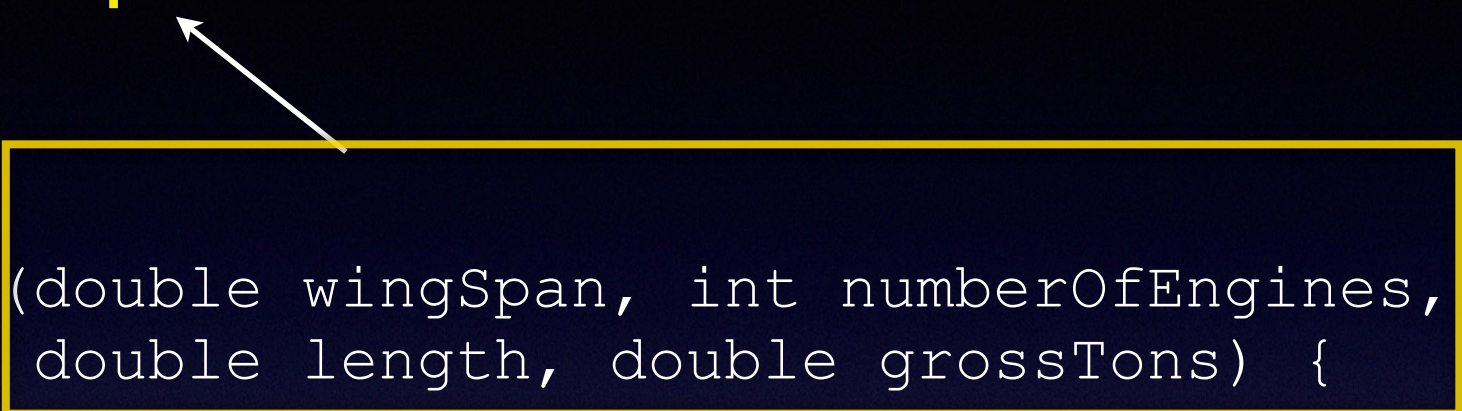
Declaración del método

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
    //do the calculation here  
    double length, double grossTons) {  
}
```


Declaración de métodos

Ejemplo

Lista de parámetros



Declaración del método

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
    //do the calculation here  
    double length, double grossTons) {  
}
```

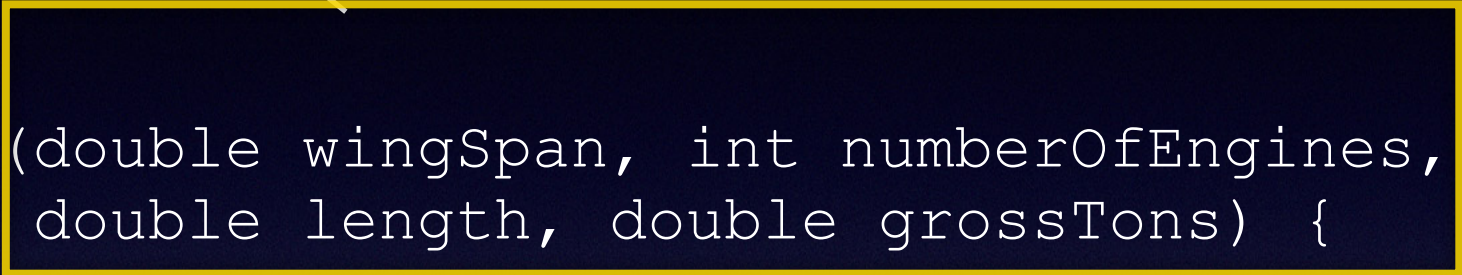
Invocación del método

```
double miValue;  
double wingLenght1 = 120.3;  
myValue = 3.5 * calculateAnswer(wingLenght1, 4, 100.5, 345.8);
```


Declaración de métodos

Ejemplo

Lista de parámetros

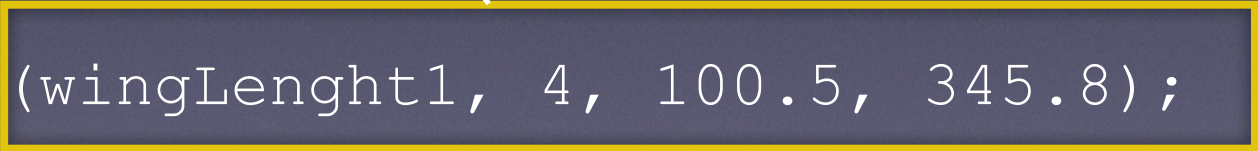


Declaración del método

```
public double calculateAnswer(double wingSpan, int numberOfEngines,  
    //do the calculation here  
    double length, double grossTons) {  
}
```

Invocación del método

Lista de argumentos



```
double miValue;  
double wingLenght1 = 120.3;  
myValue = 3.5 * calculateAnswer(wingLenght1, 4, 100.5, 345.8);
```


Declaración e invocación de métodos

Ejemplo

```
class MyClass {  
    public int calculateProduct(int a, int b) {  
        return a * b;  
    }  
    private int calculateProduct2(int a, int b){  
        return a * b;  
    }  
    public int calculateProduct3(int a, int b) {  
        return calculateProduct2(a, b);  
    }  
}
```


Declaración e invocación de métodos

Ejemplo

```
public class Methods {  
    public static void main(String[] argv) {  
        int i, j;  
        MyClass myObject = new MyClass();  
        i = 4;  
        j = 5;  
        System.out.println(myObject.calculateProduct(i, j));  
        System.out.println(myObject.calculateProduct(3, 6));  
        System.out.println(myObject.calculateProduct(i, 6) * 2);  
        System.out.println(myObject.calculateProduct3(3, 6));  
    }  
}
```

```
class MyClass {  
    public int calculateProduct(int a, int b) {  
        return a * b;  
    }  
    private int calculateProduct2(int a, int b){  
        return a * b;  
    }  
    public int calculateProduct3(int a, int b) {  
        return calculateProduct2(a, b);  
    }  
}
```


Sobrecarga de métodos

```
public class DataArtist {  
    ...  
    public void draw(String s) {  
        ...  
    }  
    public void draw(int i) {  
        ...  
    }  
    public void draw(double f) {  
        ...  
    }  
    public void draw(int i, double f) {  
        ...  
    }  
}
```


Constructores

```
public Bicycle(int startCadence, int startSpeed, int startGear) {  
    gear = startGear;  
    cadence = startCadence;  
    speed = startSpeed;  
}
```


Constructores

```
public Bicycle(int startCadence, int startSpeed, int startGear) {  
    gear = startGear;  
    cadence = startCadence;  
    speed = startSpeed;  
}
```

```
Bicycle myBike = new Bicycle(30, 0, 8);
```


Constructores

```
public Bicycle(int startCadence, int startSpeed, int startGear) {  
    gear = startGear;  
    cadence = startCadence;  
    speed = startSpeed;  
}
```

```
Bicycle myBike = new Bicycle(30, 0, 8);
```

```
public Bicycle() {  
    gear = 1;  
    cadence = 10;  
    speed = 0;  
}
```


Constructores

```
public Bicycle(int startCadence, int startSpeed, int startGear) {  
    gear = startGear;  
    cadence = startCadence;  
    speed = startSpeed;  
}
```

```
Bicycle myBike = new Bicycle(30, 0, 8);
```

```
public Bicycle() {  
    gear = 1;  
    cadence = 10;  
    speed = 0;  
}
```

```
Bicycle yourBike = new Bicycle();
```


Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

- 5.- Crea la clase Point con 2 atributos int, **x** e **y** con 2 constructores:
 - Uno sin parámetros que inicialice x e y a 0
 - El segundo tomará 2 parámetros de tipo int
- 6.- Crea el método toString() que devuelve el punto de la forma “(x , y)”
- 7.- Crea el método moveTo con 2 parámetros para poder mover el punto a la posición indicada.
- 8.- Crea una clase Main (con un método main) que declare un punto y luego lo mueva a otra posicionales para finalmente, imprimir su valor llamando al método toString().
- 9.- Asegúrate de que tu clase encapsulada sus datos.
- 10.- Crea los métodos getX() y getY()
- 11.- Crea los métodos setX() y setY()
- 12.- Crea el método setOffset(int offX, int offY) que incrementa los valores de x e y.
- 13.- Crea Main2 e instancia dos Points (4,5) y (6,8).
- 14.- muévelos 4 unidades a la derecha y 4 unidades hacia arriba (offset x = 4, offset y = 4)
- 15.- Imprimelos

Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

- 5.- Crea la clase Point con 2 atributos int, **x** e **y** con 2 constructores:
- Uno sin parámetros que inicialice x e y a 0
 - El segundo tomará 2 parámetros de tipo int

```
public class Point {  
    public int x;  
    public int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

- 5.- Crea la clase Point con 2 atributos int, **x** e **y** con 2 constructores:
- Uno sin parámetros que inicialice x e y a 0
 - El segundo tomará 2 parámetros de tipo int

```
public class Point {  
    public int x;  
    public int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.setX(12);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

```
public class Point {  
    public int x;  
    public int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.setX(12);  
    }  
}
```

Imagina la siguiente restricción: No queremos valores negativos para x e y.

Nada evita que alguien que haga uso de nuestra clase Point haga `p.x = -5`;

Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.x = 12;  
    }  
}
```

Esto genera un error ahora



Cómo resolverlo: Haz los atributos **private**. Esta es una buena regla general:

SIEMPRE, haz los atributos private (excepto las constantes),
Y cámbialos a public, por ejemplo, más adelante si lo necesitas.

Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
  
    public void setX(int newX) {  
        if (newX < 0) {  
            x = -newX;  
        } else {  
            x = newX;  
        }  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.setX(12);  
    }  
}
```

Para cambiar el valor de x necesito un método public

Hacemos los ejercicios del 5 al 15 con ayuda

Crea un nuevo proyecto Java “Geometry”

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
  
    public Point() {  
        x = 0;  
        y = 0;  
    }  
  
    public void setX(int newX) {  
        x = newX;  
    }  
    public void setY(int newY) {  
        y = newY;  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.setX(12);  
    }  
}
```

Simplifiquemos:

this This significa esta instancia

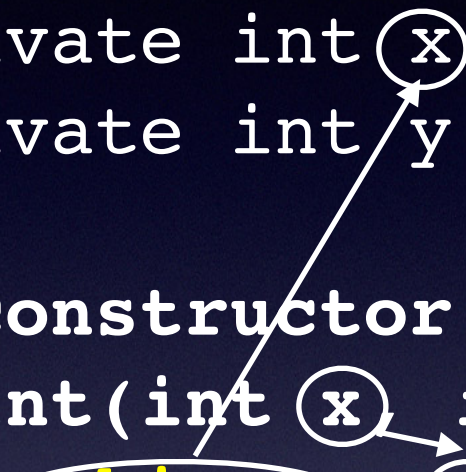
Usando this con un atributo

```
public class Point {  
    private int x;  
    private int y;  
  
    //constructor  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public setX(int x) {  
        this.x = x;  
    }  
}
```


this

Usando this con un constructor

```
public class Point {  
    private int x;  
    private int y;  
  
    //constructor  
    Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    //constructor  
    Point() {  
        this(0,0);  
    }  
}
```



Hacemos los ejercicios del 5 al 15 con ayuda

En nuestro ejemplo:

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
}
```

Podemos instanciar un punto y hacer:

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.setX(12);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

-6.- Crea el método toString() que devuelve el punto de la forma “(x , y)”

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

7.- Crea el método moveTo con 2 parámetros para poder mover el punto a la posición indicada.

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```

```
    public void moveTo(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

8.- Crea una clase Main (con un método main) que declare un punto y luego lo mueva a otra posicionales para finalmente, imprimir su valor llamando al método toString().

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```

```
    public void moveTo(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Crea un nuevo fichero Main.java

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.moveTo(8, 9);  
        System.out.println(p);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

9.- Asegúrate de que tu clase encapsula sus datos..

```
public class Point {  
    private int x;  
    private int y;
```

Esto significa que compruebes que tus atributos son private. Lo son.

```
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }
```

```
    public Point() {  
        this(0, 0);  
    }
```

```
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }
```

```
    public String toString() {  
        return "(" + x + ", " + y + " )";  
    }
```

```
    public void moveTo(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }
```

```
    public class Main {  
        public static void main(String[] argv) {  
            Point p = new Point(3, 4);  
            p.moveTo(8, 9);  
            System.out.println(p);  
        }  
    }
```


Hacemos los ejercicios del 5 al 15 con ayuda

10.- Crea los métodos getX() y getY()

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```

```
    public void moveTo(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public int getY() {  
        return y;  
    }  
}
```

```
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.moveTo(8, 9);  
        System.out.println(p);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

11.- Crea los métodos setX() y setY()

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```

Ya está hecho

```
public void moveTo(int x, int y) {  
    this.x = x;  
    this.y = y;  
}  
  
    public int getX() {  
        return x;  
    }  
  
    public int getY() {  
        return y;  
    }  
}  
  
public class Main {  
    public static void main(String[] argv) {  
        Point p = new Point(3, 4);  
        p.moveTo(8, 9);  
        System.out.println(p);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

12.- Crea el método `setOffset(int offX, int offY)` que incrementa los valores de `x` e `y`.

```
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Point() {
        this(0, 0);
    }

    public void setX(int x) {
        this.x = x;
    }
    public void setY(int y) {
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

```
    public void moveTo(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }

    public void setOffset(int offsetX, int offsetY) {
        x += offsetX;
        y += offsetY;
    }
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

13.- Crea Main2 e instancia dos Points (4,5) y (6,8).

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Point() {  
        this(0, 0);  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
    public void setY(int y) {  
        this.y = y;  
    }  
  
    public String toString() {  
        return "(" + x + ", " + y + ")";  
    }  
}
```

```
    public void moveTo(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public int getY() {  
        return y;  
    }  
  
    public void setOffset(int offsetX, int offsetY) {  
        x += offsetX;  
        y += offsetY;  
    }  
}
```

```
public class Main2 {  
    public static void main(String[] argv) {  
        Point p1 = new Point(4, 5);  
        Point p2 = new Point(6, 8);  
    }  
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

14.- muévelos 4 unidades a la derecha y 4 unidades hacia arriba (offset x = 4, offset y = 4)

```
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Point() {
        this(0, 0);
    }

    public void setX(int x) {
        this.x = x;
    }

    public void setY(int y) {
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}

public void moveTo(int x, int y) {
    this.x = x;
    this.y = y;
}

public int getX() {
    return x;
}

public int getY() {
    return y;
}

public void setOffset(int offsetX, int offsetY) {
    x += offsetX;
    y += offsetY;
}

public class Main2 {
    public static void main(String[] argv) {
        Point p1 = new Point(4, 5);
        Point p2 = new Point(6, 8);
        p1.setOffset(4, 4);
        p2.setOffset(4, 4);
    }
}
```


Hacemos los ejercicios del 5 al 15 con ayuda

15.- Imprimelos

```
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Point() {
        this(0, 0);
    }

    public void setX(int x) {
        this.x = x;
    }

    public void setY(int y) {
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public void moveTo(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }

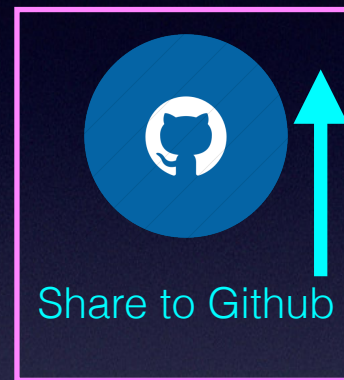
    public int getY() {
        return y;
    }

    public void setOffset(int offsetX, int offsetY) {
        x += offsetX;
        y += offsetY;
    }
}

public class Main2 {
    public static void main(String[] argv) {
        Point p1 = new Point(4, 5);
        Point p2 = new Point(6, 8);
        p1.setOffset(4, 4);
        p2.setOffset(4, 4);
        System.out.println("P1 = " + p1.toString());
        System.out.println("P2 = " + p1.toString());
    }
}
```


Hemos hecho los ejercicios 5 al 15 con ayuda

Comparte tu proyecto Geometry en Github



Sube la URL a la tarea individual
de Aules

Haz los ejercicios del 16 al 29

Estos los haces por tu cuenta

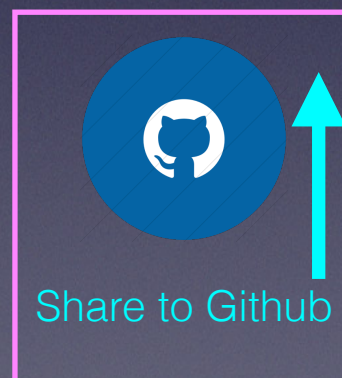
Haz un commit por cada ejercicio (incluye tu nombre en el texto del commit)

- 16.- Crea la clase Segment con 2 puntos (startPoint y endPoint). Tendrá 2 constructores: sin argumentos, creará 2 puntos (0, 0).
- 17.- Otro constructor recibirá 2 puntos.
- 18.- Crea el método getModule que devuelve un valor double con la distancia entre ambos puntos.
- 19.- Crea el método toString que imprime el segmento (x, y) - (x, y).
- 20.- Crea el método setOffset(int offX, int offY) que mueve el segmento dados esos desplazamientos.
- 21.- Crea los métodos setStartPoint y setEndPoint.
- 22.- Crea el ejemplo Main3 para crear un Segment con los puntos (4, 5) y (6, 8), moverlo 4, 4 de desplazamiento y luego imprimirlo. Imprime también su módulo.
- 23.- En la clase Segment, crea los métodos getStartPoint() y getEndPoint().

Haz los ejercicios del 16 al 29

Haz un commit por cada ejercicio (incluye tu nombre en el texto del commit)

- 24.- Crea la clase Polygon que almacenará un conjunto de puntos. Crea sus atributos (nota: usa un array de puntos para almacenar los puntos del polígono).
- 25.- Crea un constructor que reciba un conjunto de puntos.
- 26.- Crea un constructor sin parámetros.
- 27.- Crea el método toString que devuelve: (a, b) - (c, d) - (e, f) - (g, h) - (a, b)
- 28.- Crea el método getLength que devuelve la longitud del polígono.
- 29.- Crea el método setOffset. Crea un MainPolygon con la función main() para probar los métodos que has creado.



Sube la URL a la tarea individual de Aules

Passing an undefined number of arguments

```
public Polygon(Point... ps) {  
    points = ps;  
}
```


Passing an undefined number of arguments

```
public Polygon(Point... ps) {  
    points = ps;  
}
```

```
Point point1 = new Point(5, 8);
```

```
Point point2 = new Point(6, 12);
```

```
Point point3 = new Point(1, 4);
```

```
myPolygon = new Polygon(point1, point2, point3);
```


Passing an undefined number of arguments

```
public Polygon(Point... ps) {  
    points = ps;  
}
```

```
Point point1 = new Point(5, 8);  
Point point2 = new Point(6, 12);  
Point point3 = new Point(1, 4);
```

```
myPolygon = new Polygon(point1, point2, point3);
```

Equivalente

```
Point[] myPoints = new Point[3];  
myPoints[0] = new Point(5, 8);  
myPoints[1] = new Point(6, 12);  
myPoints[2] = new Point(1, 4);  
  
myPolygon = new Polygon(myPoints);
```


Passing primitive data type arguments

```
public class PassPrimitiveByValue {  
    public static void main(String[] args) {  
        int x = 3;  
  
        // invoke passMethod() with  
        // x as argument  
        passMethod(x);  
  
        // print x to see if its  
        // value has changed  
        System.out.println("After invoking passMethod, x = " + x);  
    }  
  
    // change parameter in passMethod()  
    public static void passMethod(int p) {  
        p = 10;  
    }  
}
```


Passing primitive data type arguments

```
public class PassPrimitiveByValue {  
    public static void main(String[] args) {  
        int x = 3;  
  
        // invoke passMethod() with  
        // x as argument  
        passMethod(x);  
  
        // print x to see if its  
        // value has changed  
        System.out.println("After invoking passMethod, x = " + x);  
    }  
  
    // change parameter in passMethod()  
    public static void passMethod(int p) {  
        p = 10;  
    }  
}
```

Después de invocar passMethod, x = 3

Passing reference data type arguments

```
public void movePoint(Point point, int deltaX, int deltaY) {  
    // code to move origin of circle to x+deltaX, y+deltaY  
    point.setX(point.getX() + deltaX);  
    point.setY(point.getY() + deltaY);  
}
```

```
Point p = new Point(5, 5);  
movePoint(p, 3, 3)
```


Passing reference data type arguments

```
public void movePoint(Point point, int deltaX, int deltaY) {  
    // code to move origin of circle to x+deltaX, y+deltaY  
    point.setX(point.getX() + deltaX);  
    point.setY(point.getY() + deltaY);  
}
```

```
Point p = new Point(5, 5);  
movePoint(p, 3, 3)
```

Después de movePoint

(8,8)

Usando objetos

```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

```
Point p1 = new Point(1,3);
```

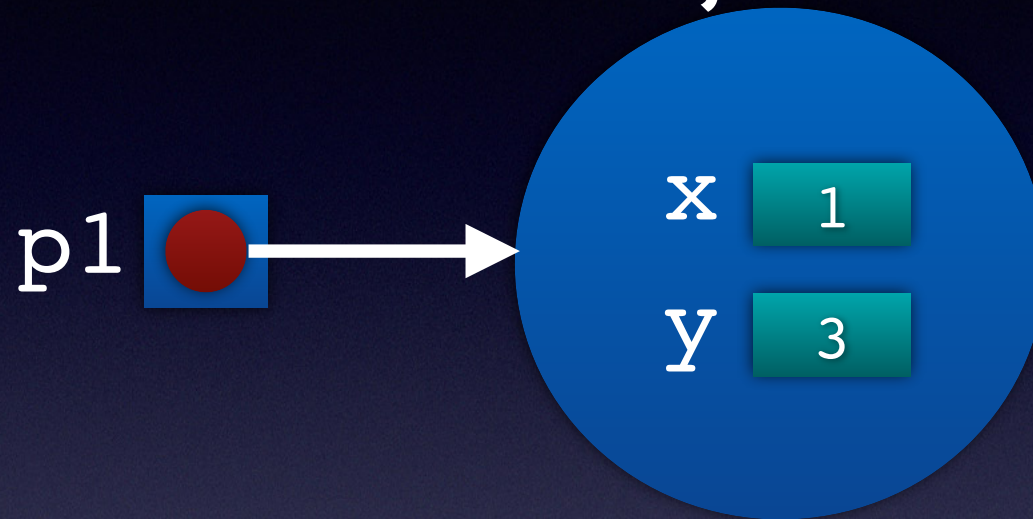

Usando objetos

```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

```
Point p1 = new Point(1,3);
```

Un objeto Point



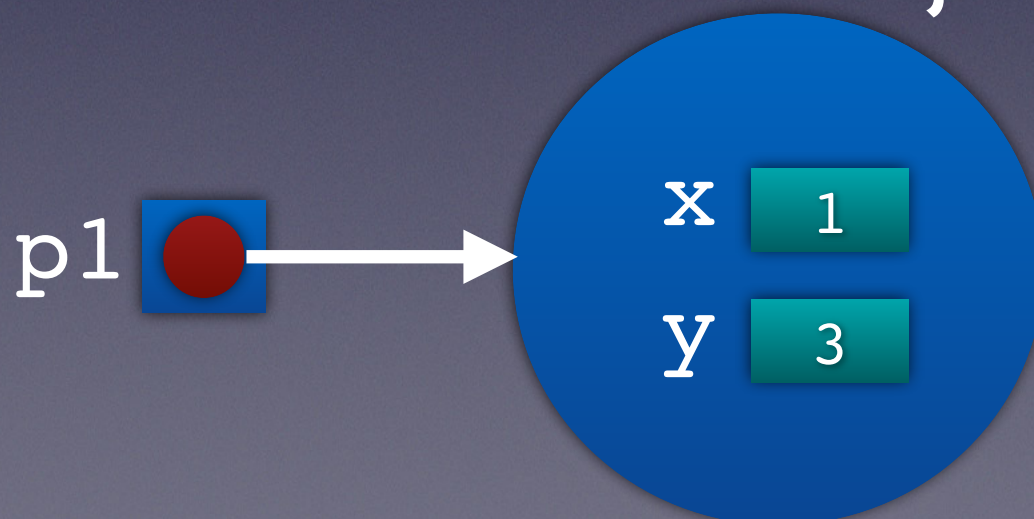
Usando objetos

```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

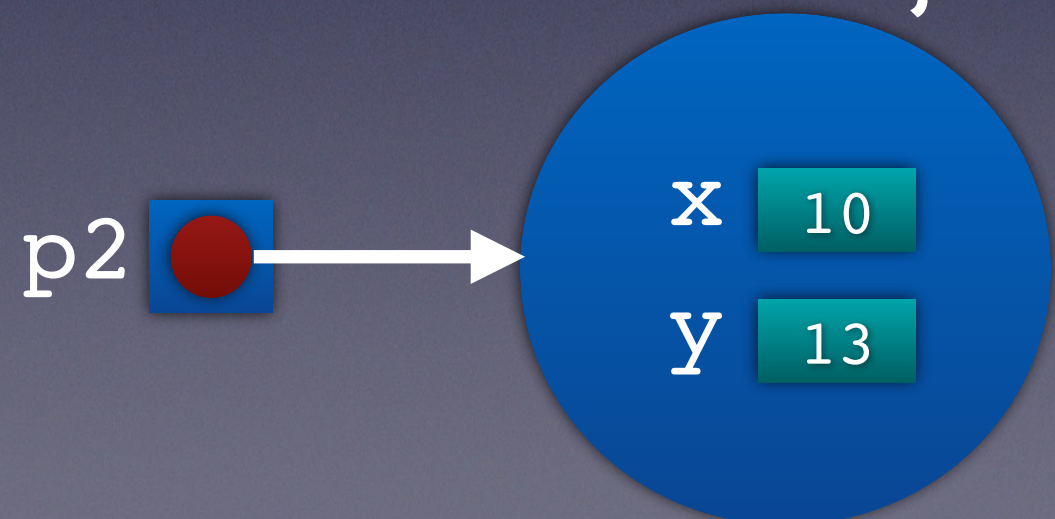
```
Point p1 = new Point(1,3);  
Point p2 = new Point(10,13);
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

a Point object



a Point object



Using objects

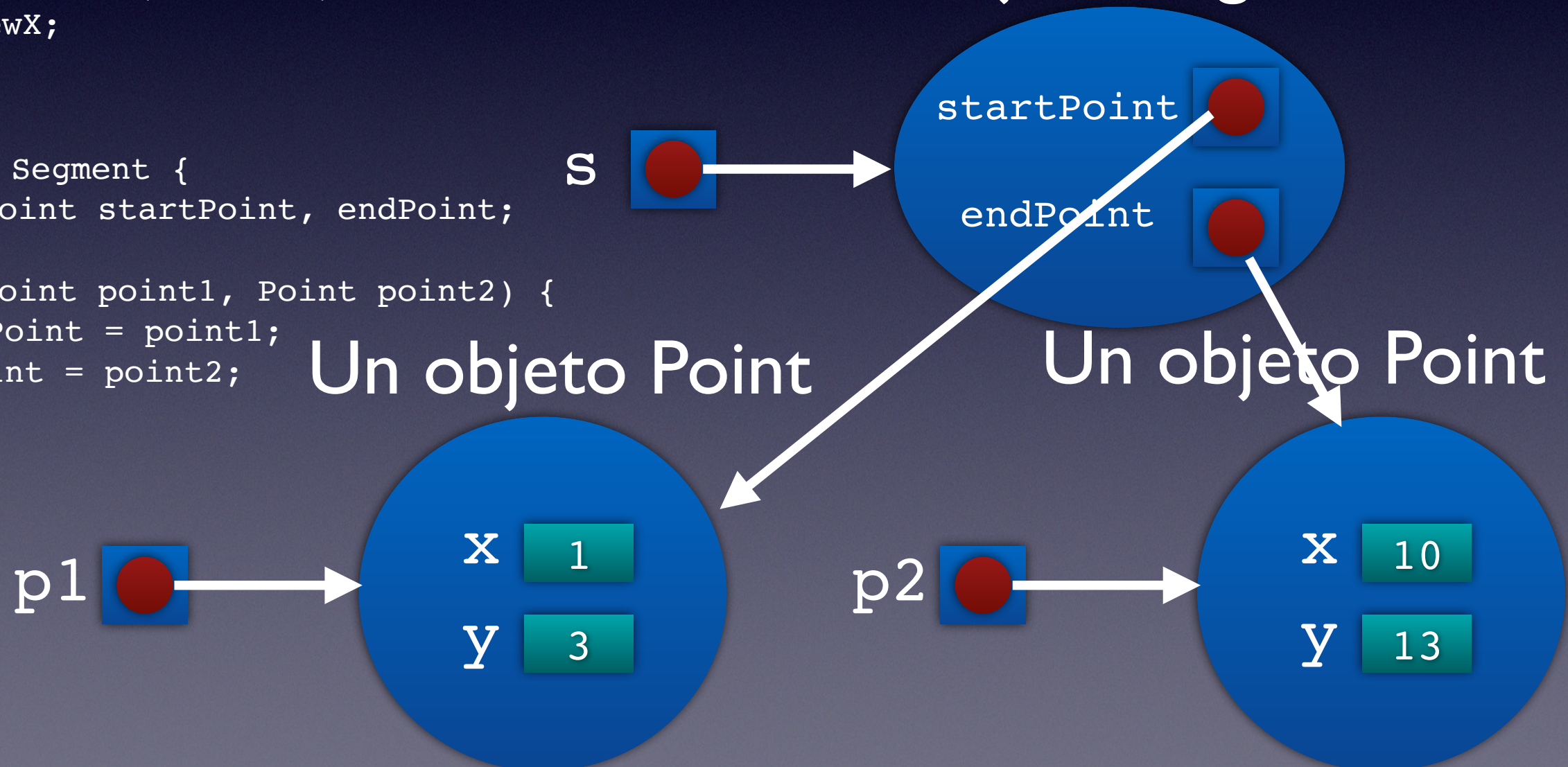
```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

```
Point p1 = new Point(1,3);  
Point p2 = new Point(10,13);
```

```
Segment s = new Segment(p1,p2);
```

Un objeto Segment



Using objects

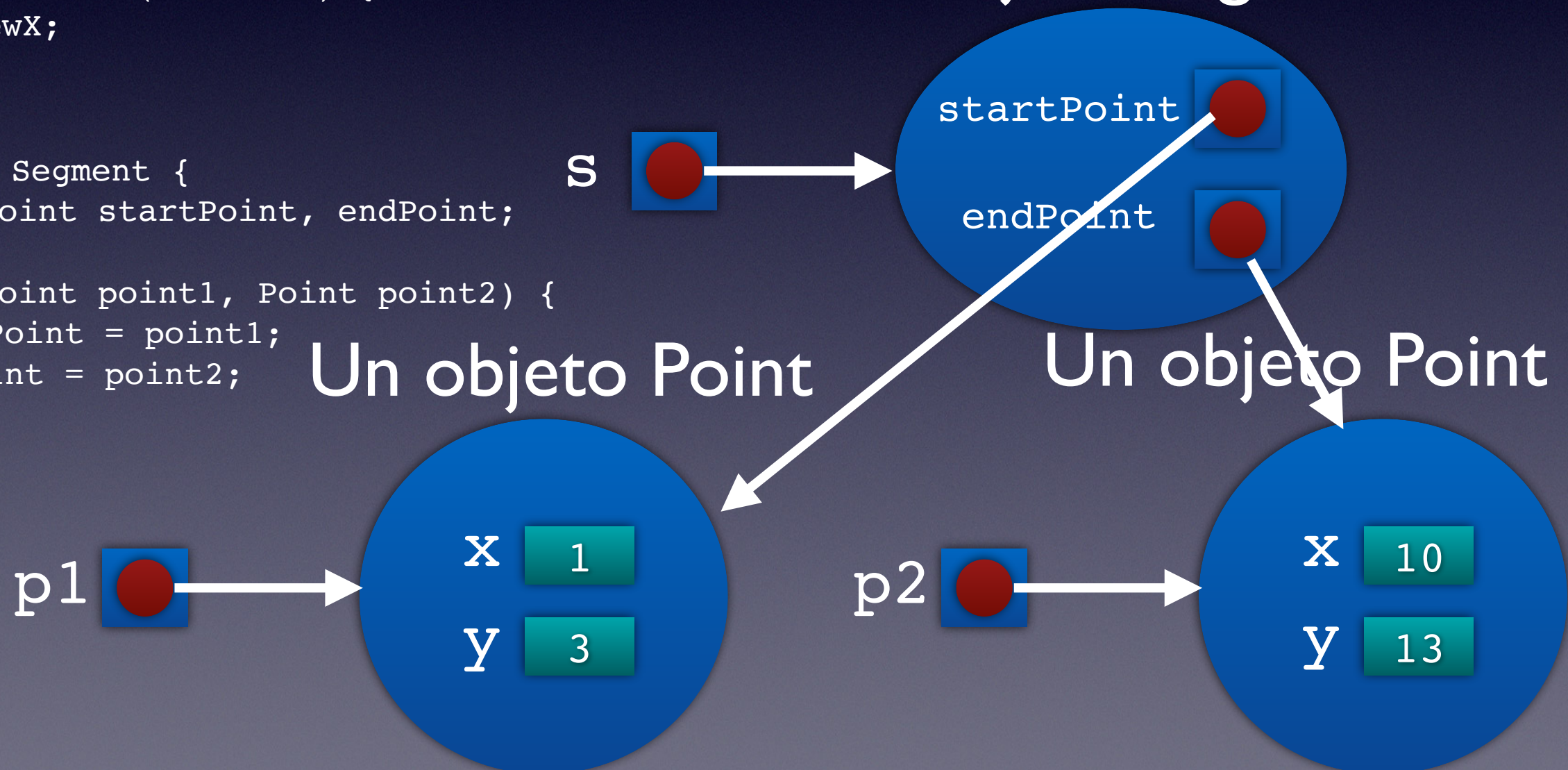
```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

```
Point p1 = new Point(1,3);  
Point p2 = new Point(10,13);
```

```
Segment s = new Segment(p1,p2);  
p1 = setX(50);
```

Un objeto Segment



Using objects

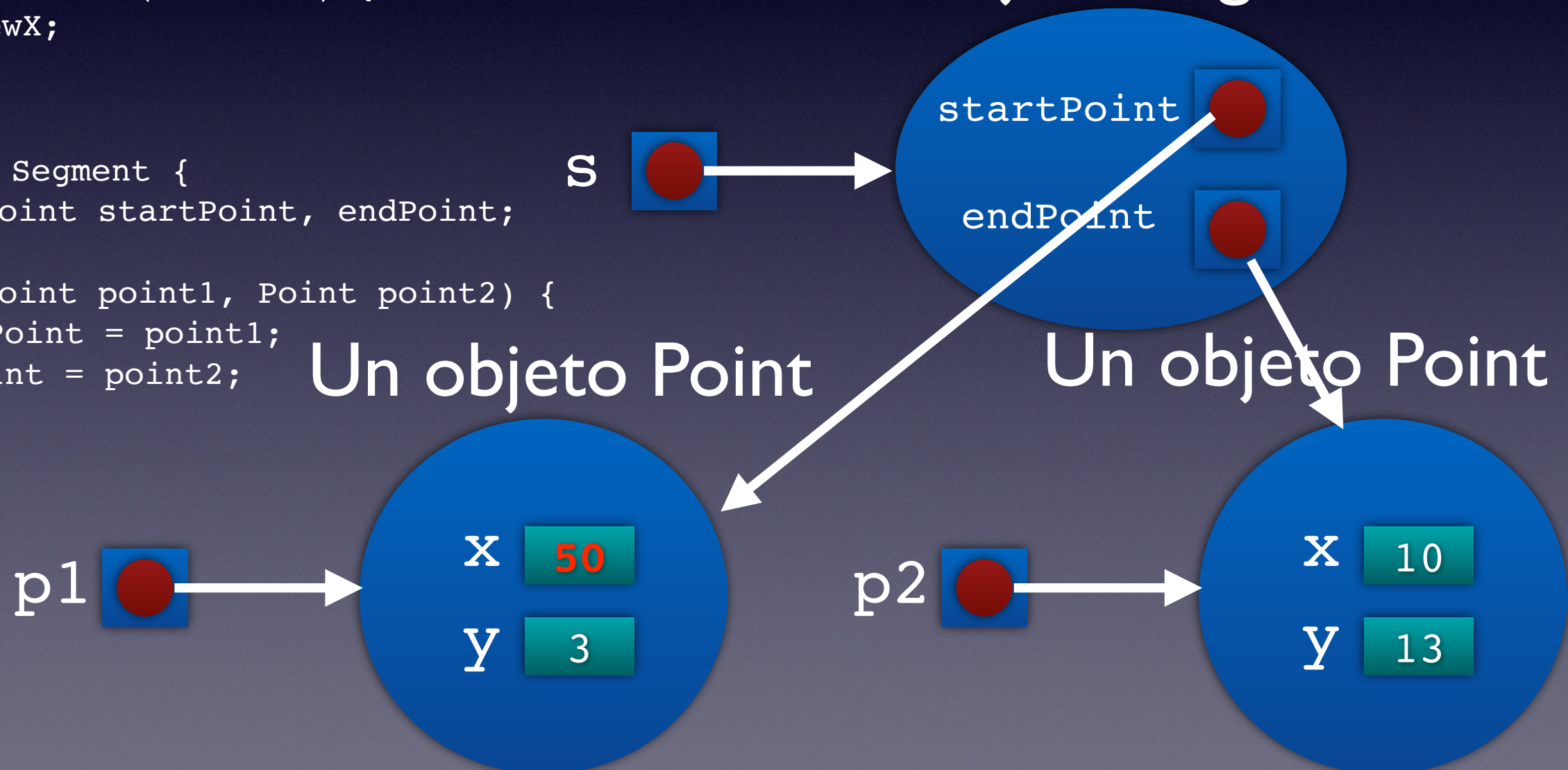
```
public class Point {  
    private int x, y;  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

```
public class Segment {  
    private Point startPoint, endPoint;  
  
    Segment(Point point1, Point point2) {  
        startPoint = point1;  
        endPoint = point2;  
    }  
}
```

```
Point p1 = new Point(1,3);  
Point p2 = new Point(10,13);
```

```
Segment s = new Segment(p1,p2);  
p1 = setX(50);
```

Un objeto Segment



Haz los ejercicios del 30 al 51

Trabaja en tu proyecto **Geometry**. Haz un commit por cada ejercicio (incluye tu nombre en el texto del commit)

30.- Crea la clase Rectangle como subclase de Point. El punto será la esquina inferior izquierda.

Añadirás los atributos width y height como enteros.

31.- Agrega el constructor Rectangle(). Inicializará su punto llamando a la superclase con super() y luego establecerá los valores de ancho y alto a 0.

32.- Crea la clase Main4. Declara un Rectangle y llama a su constructor Rectangle(). Imprímelo. Muévelo a (4, 7). Imprímelo de nuevo.

33.- Crea el método toString() que devuelve: (x, y) width: xxxxx height: xxxxxx

34.- Ejecuta Main4 de nuevo.

35.- Crea los constructores: Rectangle(Point p, int newWidth, int newHeight).

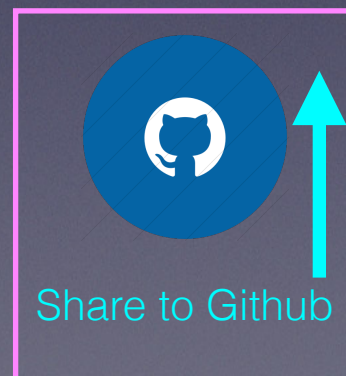
36.- Declara un rectángulo en Main4 con el punto (1,1), ancho 8 y alto 6. Imprímelo.

37.- Crea el constructor Rectangle(Point p1, Point p2). p1 es la esquina inferior izquierda y p2 la esquina superior derecha.

(Solo para estudiantes avanzados: declara un constructor Rectangle(Segment seg)).

38.- En Main4, declara un rectángulo con los puntos (2,2) y (5,8). Imprímelo.

- 39.- Crea el método `getArea()` que devuelve el área.
- 40.- Crea el método `getPoint()` que devuelve el punto (punto inferior izquierdo).
- 41.- Modifica `Main4` para obtener el punto del último rectángulo usado y luego imprímelo.
- 42.- Crea el método `getTopLeftPoint()`
- 43.- Crea el método `getTopRightPoint()`
- 44.- Crea el método `getBottomLeftPoint()` (igual que `getPoint()`)
- 45.- Crea el método `getBottomRightPoint()`
- 46.- Modifica `Main4` para declarar un rectángulo (5, 6) con ancho = 10 y alto = 8.
Imprime sus puntos así:
Arriba-Izquierda: (x, y) Arriba-Derecha: (x, y) Abajo-Izquierda: (x, y) Abajo-Derecha: (x, y)
- 47.- Agrega los métodos `getWidth`, `getHeight`, `setWidth` y `setHeight` a `Rectangle`.
- 48.- Crea la clase `Main5`. Declara e instancia un array de 10 puntos.
- 49.- Inicializa el array con los puntos (0,0), (1,1), (2,2), (3,3), etc.
- 50.- Imprímelos todos.
- 51.- Crea un array de 10 rectángulos e inicialízalos con ancho y alto = 1 y su origen en (0,0), (1,1), (2,2), etc. Puedes usar el array de puntos del ejercicio anterior si lo deseas. Imprímelo.



Sube la URL a la tarea individual de Aules

De la unidad anterior, pero usando la clase Board

1. Escribe un programa que simule el juego "BattleShip" sobre un tablero de 8x8. La computadora colocará 10 barcos al azar (de una sola celda). El usuario introducirá una coordenada (primero letra y luego un número) y el programa escribirá el tablero completo cada vez, con los barcos hundidos (X) y los disparos fallidos (círculos). Mostrará el contador de disparos y el número de barcos hundidos.

Ejemplo:

SHOTS: 12

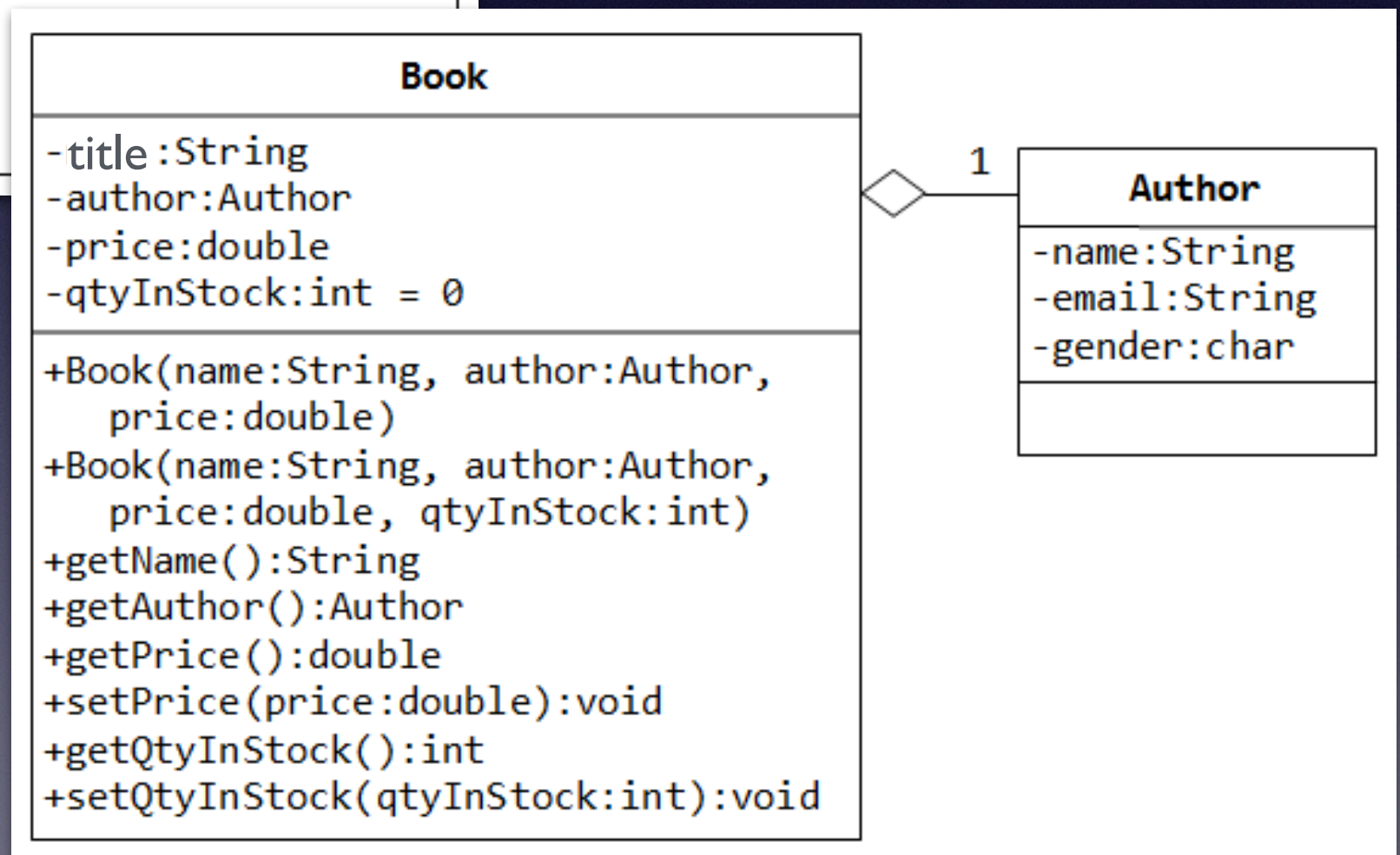
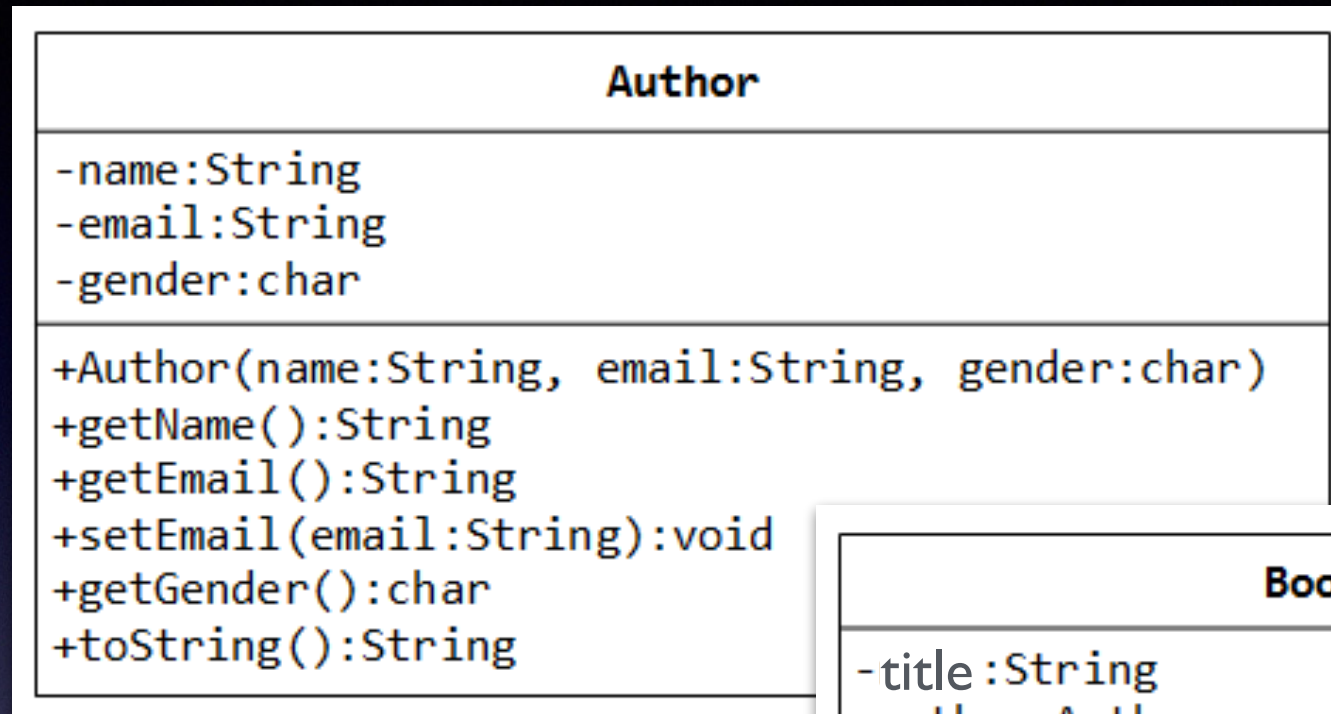
SUNK SHIPS: 3

	1	2	3	4	5	6	7	8
A								
B		O		O		O		
C		O					O	
D	X	O						
E				X	O		O	
F				O				
G	O				O		X	
H								O

Enter row (Letter): C

Enter column (Number): 2

Ejercicios



Ejercicio. Crea el proyecto Library

Ejercicio: Crea las clases `Author` y `Book`

Diseña la clase `Author` como se muestra en el diagrama de clases. Contiene:

- Tres `private` variables de instancia: `name` (`String`), `email` (`String`), y `gender` (`char` que será `'m'` or `'f'`);
- Un constructor que inicialice `name`, `email` y `gender` con los valores dados;

```
public Author (String name, String email, char gender) {.....}
```

(No habrá constructor por defecto para `Author`, ya que no hay nombres, emails o genres por defecto.)
- `public` getters/setters: `getName()`, `getEmail()`, `setEmail()`, y `getGender()`;
(No hay setters para `name` y `gender`, ya que no se pueden cambiar.)
- Un método `toString()` que devuelve "author-name (gender) at email", ej., "Tan Ah Teck (m) at ahTeck@somewhere.com".

Escribe la clase `Author`. También, escribe una programa `TestAuthor` que compruebe el constructor y los métodos `public`. Intenta cambiar el `email` de un autor, ej.,

```
Author anAuthor = new Author("Tan Ah Teck", "ahteck@somewhere.com", 'm');
System.out.println(anAuthor); // call toString()
anAuthor.setEmail("paul@nowhere.com")
System.out.println(anAuthor)
```


La clase `Book` del diagrama de clases contiene:

- cuatro `private` variables de instancia: `title` (`String`), `author` (de la clase `Author` que acabas de crear, asume que cada libro tiene solo un autor), `price` (`double`), y `qtyInStock` (`int`);
- Dos constructores: `public Book (String title, Author author, double price) {...}`
- `public Book (String title, Author author, double price, int qtyInStock) {...}`
- Métodos públicos `getTitle()`, `getAuthor()`, `getPrice()`, `setPrice()`, `getQtyInStock()`, `setQtyInStock()`.
- `toString()` que devuelve `"title' by author-name (gender) at email"`.

(Nota: el método `toString()` de `Author` devuelve `"name (gender) at email"`.)

Escribe la clase `Book` (que usa la clase `Author`). Y un programa de test llamado `TestBook` para tester el constructor y los métodos `public` de la clase `Book`. Nota: debes crear instancias de `Author` para usar como argumento en el constructor de `Book`. E.g.,

```
Author anAuthor = new Author(.....);
```

```
Book aBook = new Book("Java for dummy", anAuthor, 19.95, 1000);
```

```
// Use an anonymous instance of Author
```

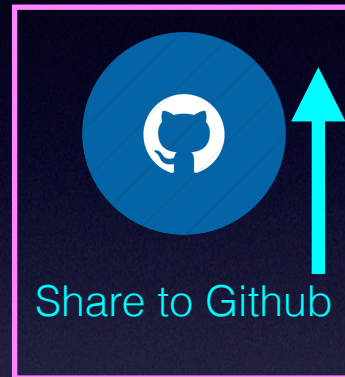
```
Book anotherBook = new Book("more Java for dummy", new Author(.....), 29.95, 888);
```

Prueba:

1. Imprimir el nombre e email de la instancia de un `Book`. (Pista: `aBook.getAuthor().getName()`, `aBook.getAuthor().getEmail()`).
2. Añade nuevos métodos `getAuthorName()`, `getAuthorEmail()`, `getAuthorGender()` en la clase `Book` que devuelvan el `name`, `email` y `gender` del autor del libro. Por ejemplo,

```
public String getAuthorName() { ..... }
```
3. Crea un array de 5 libros. Inicialízalo con algunos datos e imprímelos todos.

Comparte tu proyecto Library en Github



Sube la URL a la tarea individual
de Aules

Ejercicio. Crea el proyecto Switchboard

Escribe un programa para controlar las llamadas en una centralita telefónica.

En la centralita se registran las llamadas.

Es posible que desees usar una matriz para almacenar todas las llamadas.

Todas las llamadas tienen el número de origen de la llamada, el número de destino, la duración en segundos y la franja horaria en la que se realizó la llamada (Banda 1, 2 o 3).

La centralita tendrá un método `print()` que imprime todas las llamadas y otro `print()` que imprimirá los datos de la llamada dándole el orden de la misma.

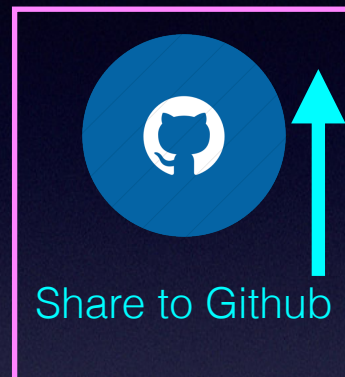
Hay dos tipos de llamadas:

- Las llamadas locales cuestan 15 céntimos por segundo. •
- Y las llamadas provinciales dependiendo de la franja horaria en la que la realicen cuestan: 20 céntimos en la banda 1, 25 céntimos en la banda 2 y 30 céntimos en la banda 3, cada segundo.

Desarrolla una clase llamada `Exercise5`, en su método principal crea una centralita, registra varios tipos y llamadas diferentes a la centralita e imprime el informe del número total de llamadas y la facturación total.

Ejercicio. Crea el proyecto Switchboard

Comparte tu proyecto Switchboard en Github

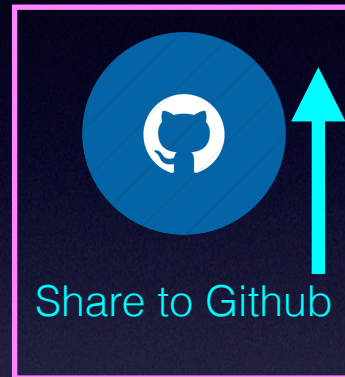


Sube la URL a la tarea individual
de Aules

Ejercicio. Crea la clase DNI con tantos constructores y métodos como necesites. Estos, por ejemplo (consúltalos desde una clase principal):

Fields	Constructors	Methods	Example/comment
int number	Dni()	getNumber()	
char letter	Dni(int n, char c)	getLetter()	set number to negative if the letter is incorrect
	Dni(int n)	setNumber()	
	Dni(String s)	toString()	—-> 18888888X
		toFormattedString()	—->18.888.888-X
		correctDni() —> boolean	false if number<0
		public static char letterForDni(int number)	
		public static String NifForDni(int number)	

Comparte tu proyecto DNI en Github



Sube la URL a la tarea individual
de Aules

Ejercicios

Escribe un programa llamado WordGuess (y una clase MagicWord) para adivinar una palabra tratando de adivinar los caracteres individuales. La palabra a adivinar se proporcionará aleatoriamente a partir de una serie de palabras disponibles. Tu programa se verá así:

Enter one character or your guess word: **t**

Attempt 1: t__t__

Enter one character or your guess word: **g**

Attempt 2: t__t__g

Enter one character or your guess word: **e**

Attempt 3: te_t__g

Enter one character or your guess word: **testing**

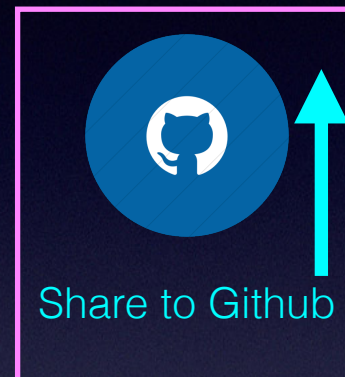
Attempt 4: Congratulation!

You got in 4 attempts

Consejos:

- Crea un array de boolean para indicar las posiciones de la palabra que se han adivinado correctamente.
- Comprueba la longitud de la entrada String para determinar si el jugador ingresa un solo carácter o una palabra adivinada. Si el jugador ingresa un solo carácter, compáralo con la palabra que se va a adivinar y actualiza el al array de boolean que mantiene el resultado hasta ahora.

Comparte tu proyecto WordGuess en Github



Sube la URL a la tarea individual
de Aules

Packages

`org.ieselcaminas.prog.util`



Packages

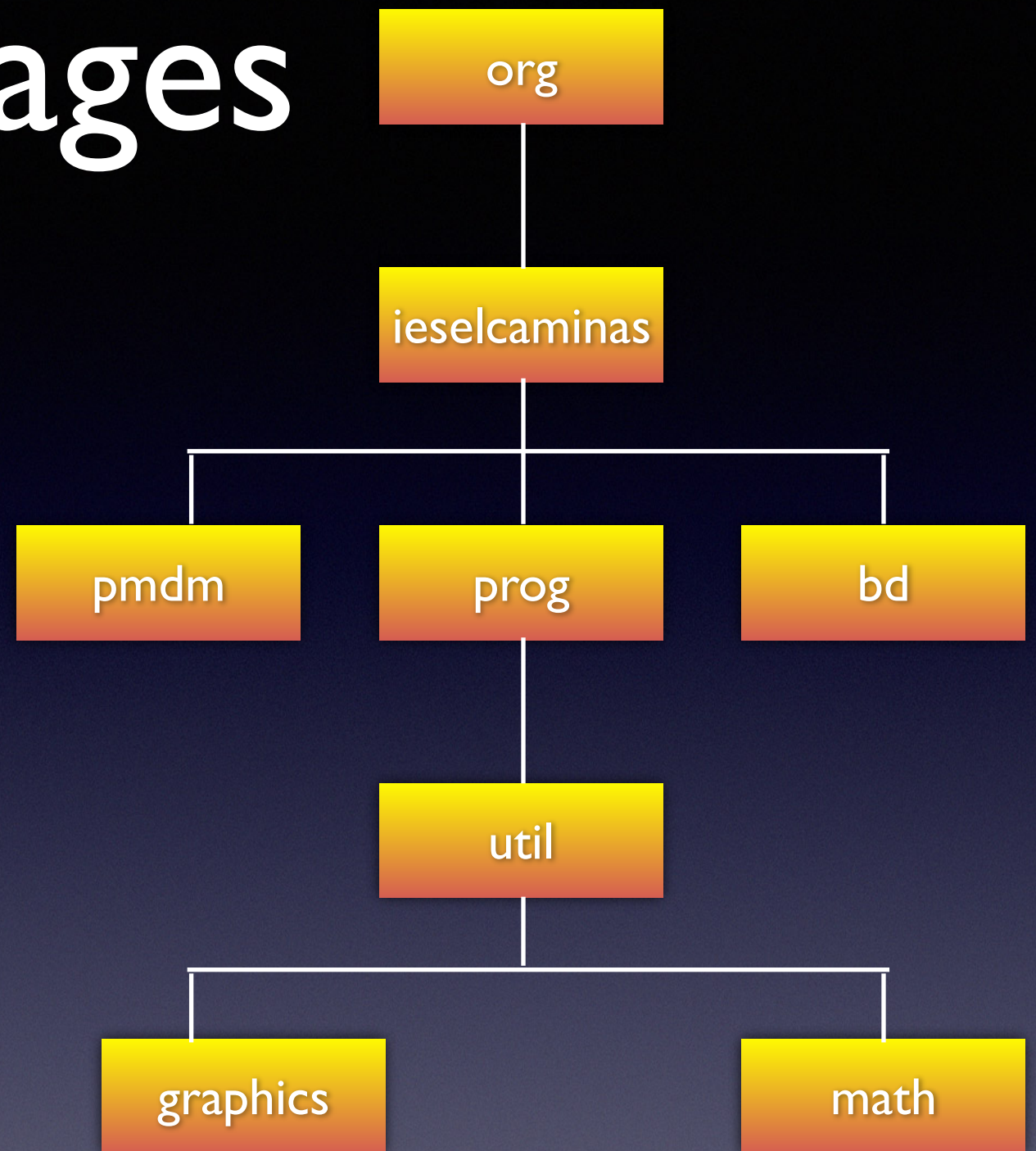
org.ieselcaminas.prog.util

org.ieselcaminas.prog.util.Dni

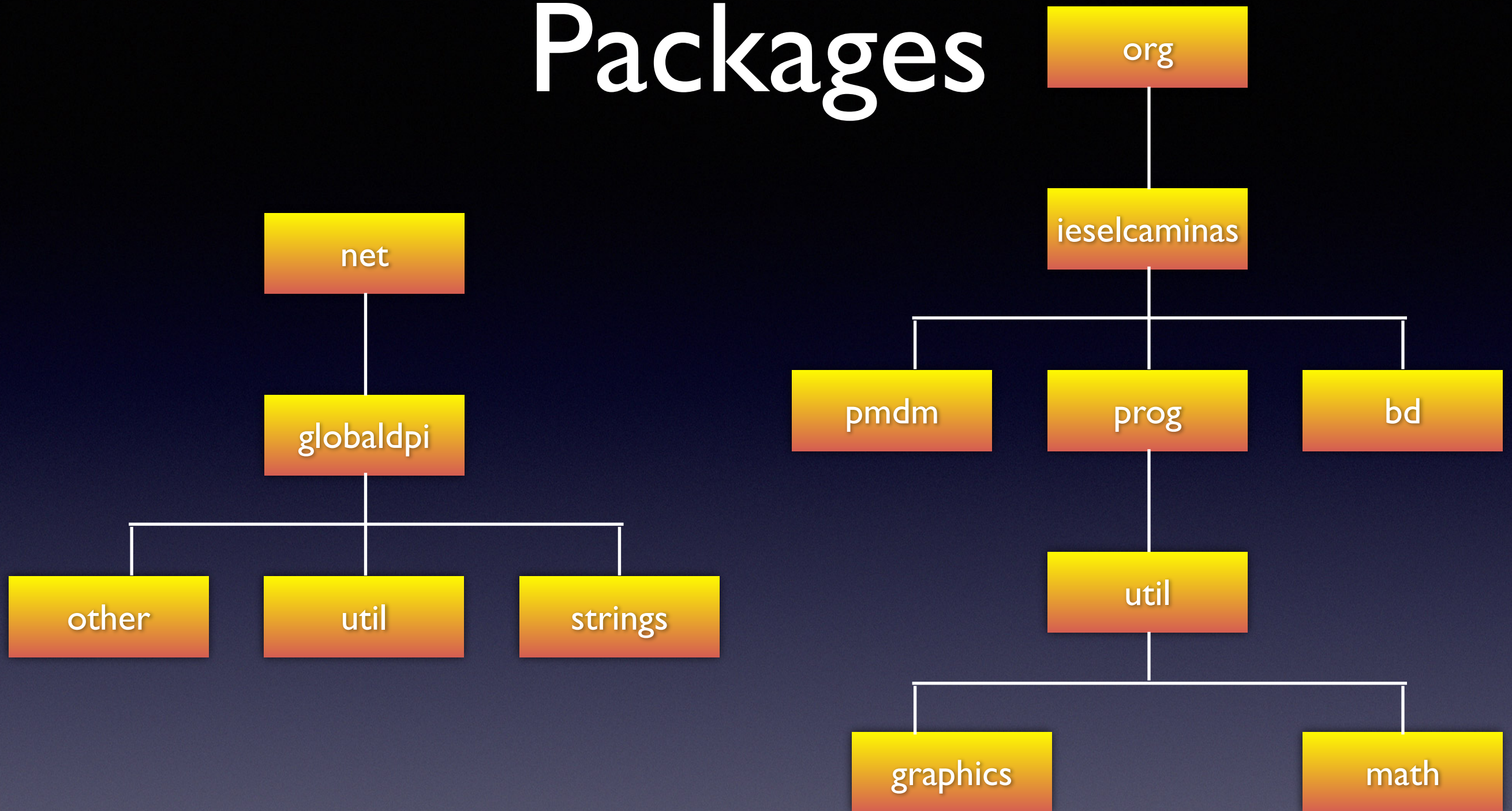
net.globaldpi.util.Dni



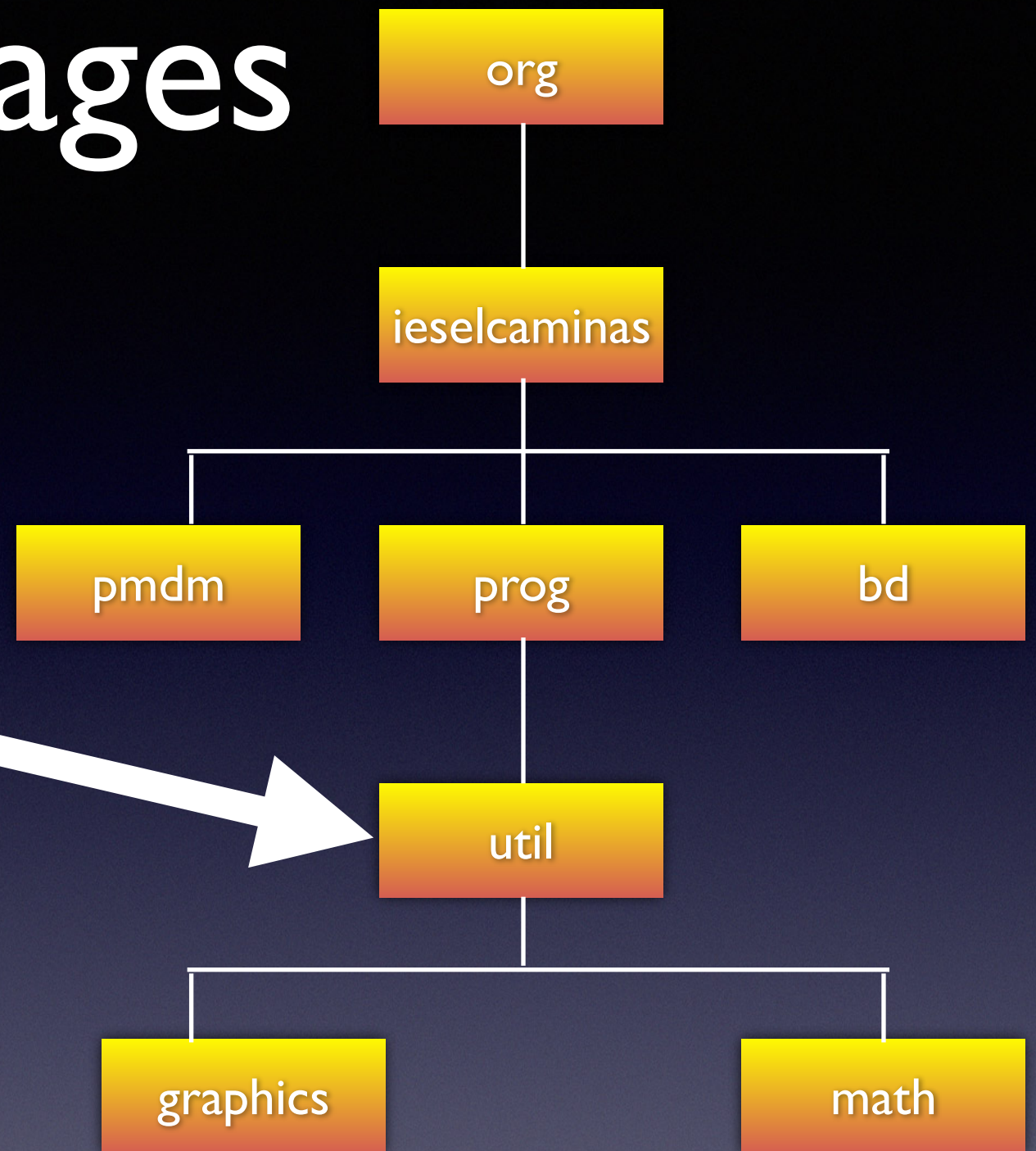
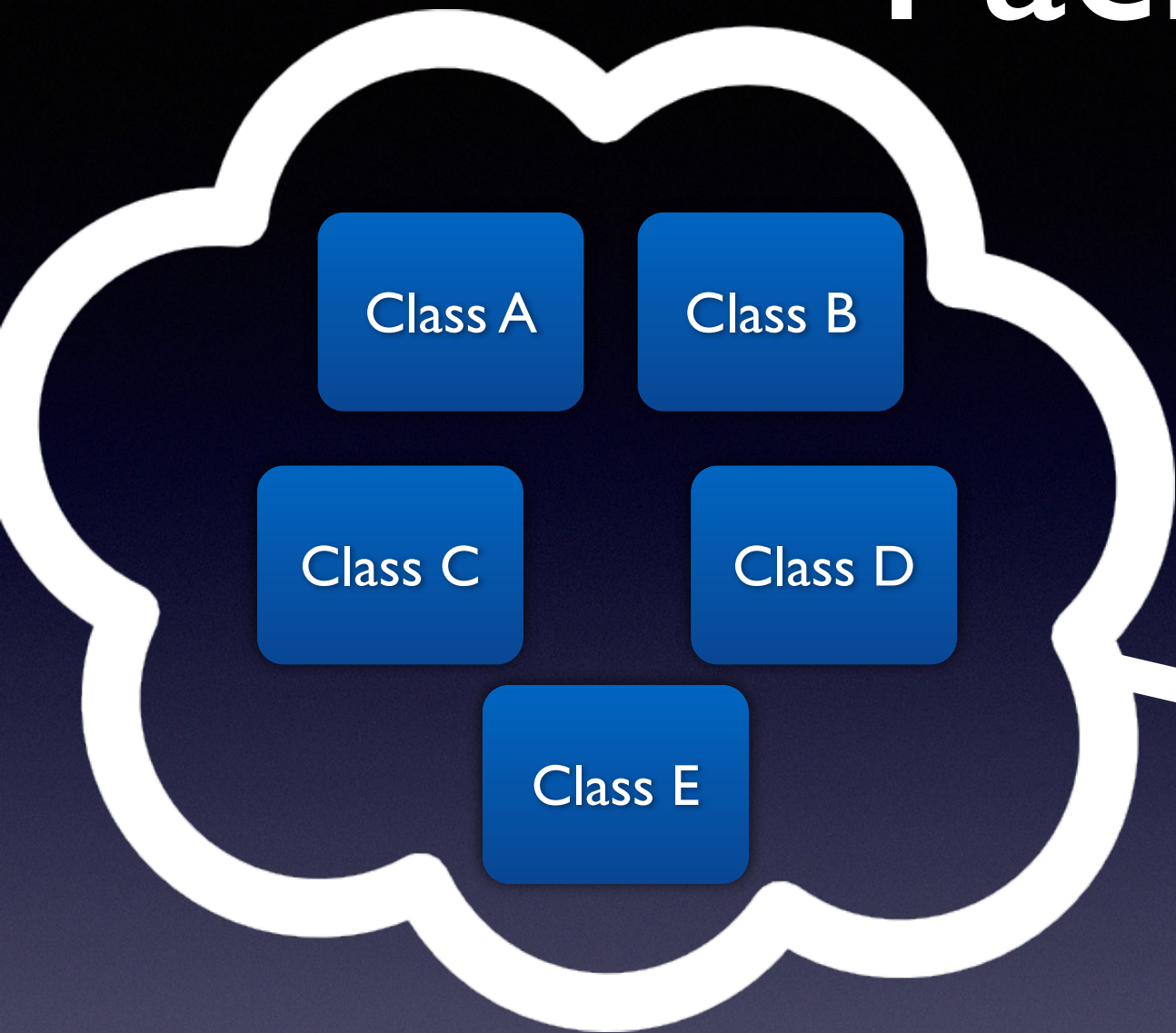
Packages



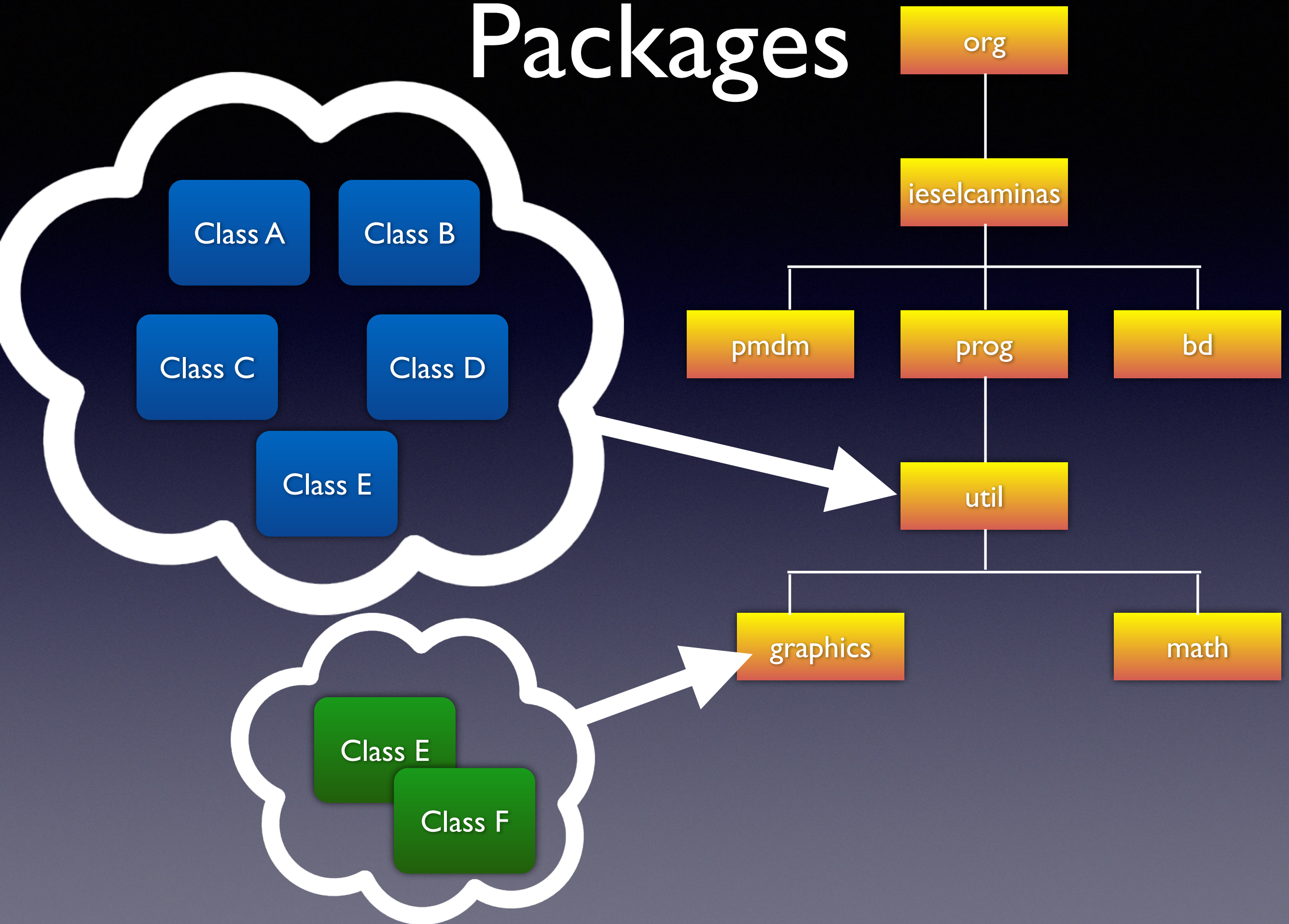
Packages



Packages



Packages



Packages

<https://docs.oracle.com/javase/8/docs/api/>

Java™ Platform
Standard Ed. 8

All Classes All Profiles

Packages

java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event
java.awt.font
java.awt.geom
java.awt.im
java.awt.im.spi
java.awt.image
java.awt.image.renderable
java.awt.print
java.beans
java.beans.beancontext
java.io

AlphaComposite
AWTEvent
AWTEventMulticaster
AWTKeyStroke
AWTPermission
BasicStroke
BorderLayout
BufferCapabilities
BufferCapabilities.FlipContents
Button
Canvas
CardLayout
Checkbox
CheckboxGroup
CheckboxMenuItem
Choice
Color
Component
ComponentOrientation
Container
ContainerOrderFocusTraversalPolicy

Ensures that the system resources held by this ZipFile object are released when there are no more references to it.

String

getComment()

Returns the zip file comment, or null if none.

ZipEntry

getEntry(String name)

Returns the zip file entry for the specified name, or null if not found.

InputStream

getInputStream(ZipEntry entry)

Returns an input stream for reading the contents of the specified zip file entry.

String

getName()

Returns the path name of the ZIP file.

int

size()

Returns the number of entries in the ZIP file.

Stream<? extends ZipEntry>

stream()

Return an ordered Stream over the ZIP file entries.

Methods inherited from class java.lang.Object

clone, equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

OPEN_READ

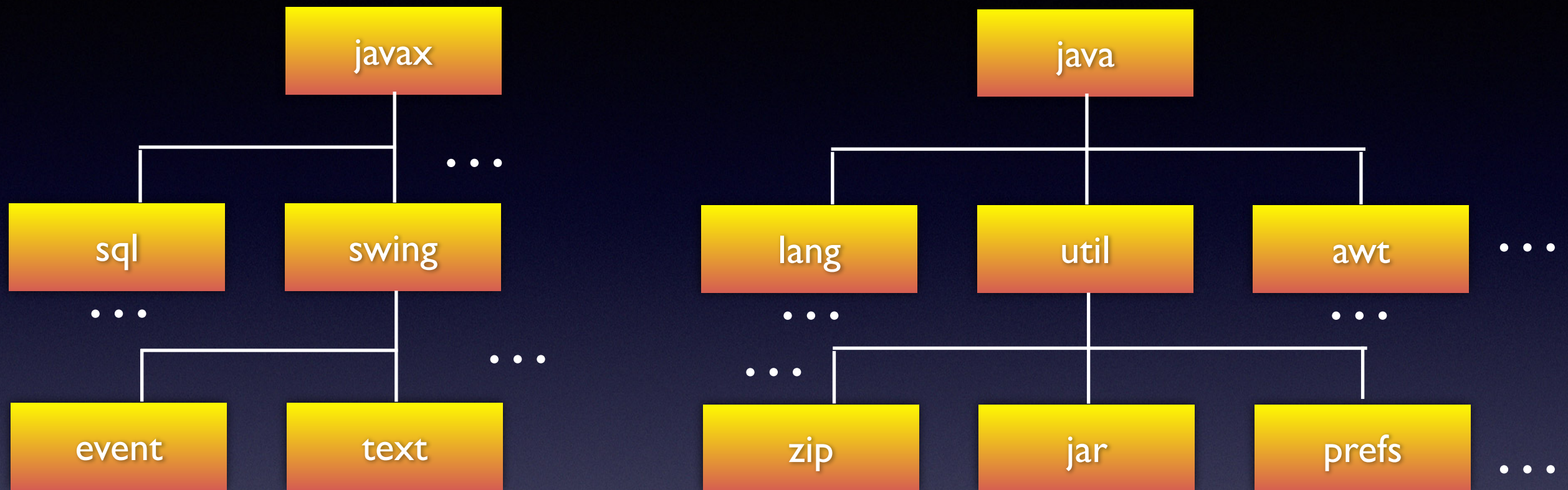
public static final int OPEN_READ

Mode flag to open a zip file for reading.

See Also:

Constant Field Values

Packages



Packages

package class method

java.lang.String.substring()

Packages

package class method
java.lang.String.substring()

package class method
org.ieselcaminas.util.Dni.getLetter()

Packages

```
package org.ieselcaminas.prog.util;  
  
public class Dni {  
    private int number;  
    private char letter;  
    ...  
}
```


Packages

```
package org.ieselcaminas.prog.util;  
  
public class Dni {  
    private int number;  
    private char letter;  
    ...  
}
```

1.- Tiene que ser la primera línea del fichero fuente

Packages

```
package org.ieselcaminas.prog.util;  
  
public class Dni {  
    private int number;  
    private char letter;  
    ...  
}
```

- 1.- Tiene que ser la primera línea del fichero fuente
- 2.- Solo una instrucción en el fichero fuente

Packages

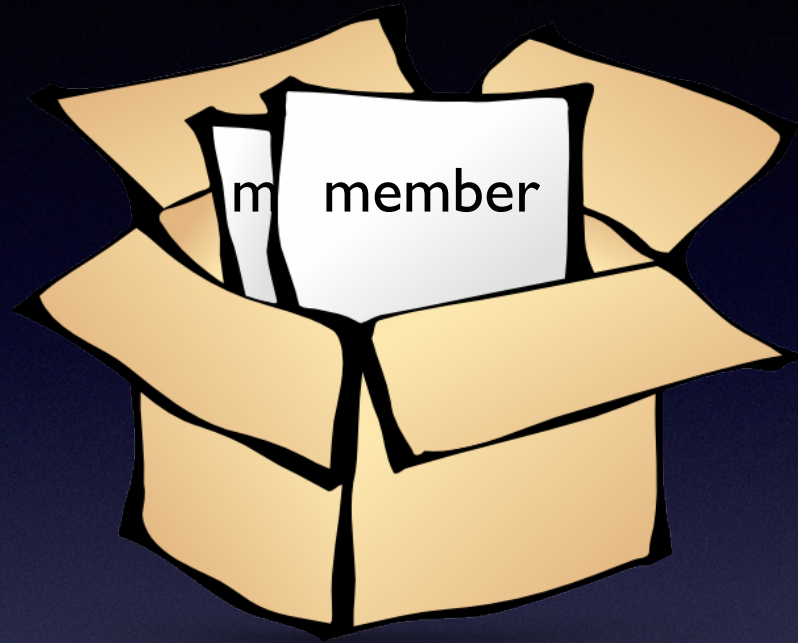
```
package org.ieselcaminas.prog.util;  
  
public class Dni {  
    private int number;  
    private char letter;  
    ...  
}
```

- 1.- Tiene que ser la primera línea del fichero fuente
- 2.- Solo una instrucción en el fichero fuente
- 3.- Se aplica a todas las clases de fichero

Usando miembros del package



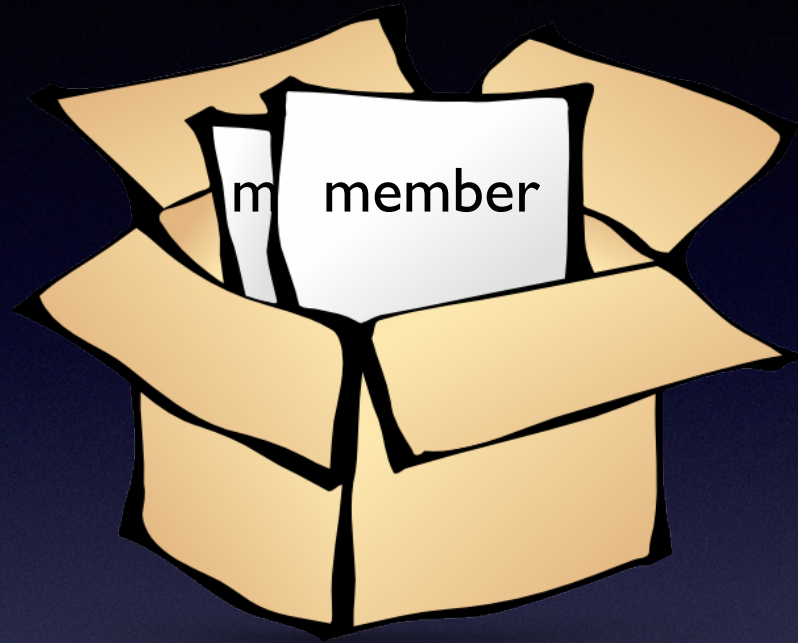
Usando miembros del package



- Fully qualified name

```
org.ieselcaminas.prog.util.Dni myDni;  
myDni = new org.ieselcaminas.prog.util.Dni(1821828);
```


Usando miembros del package



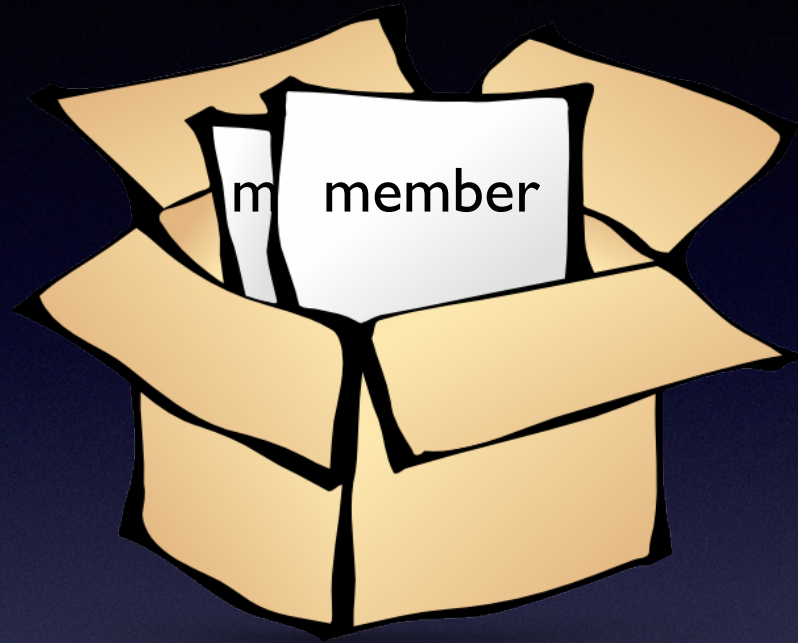
- Fully qualified name

```
org.ieselcaminas.prog.util.Dni myDni;  
myDni = new org.ieselcaminas.prog.util.Dni(1821828);
```

- Importando el miembro del package

```
import org.ieselcaminas.prog.util.Dni;  
Dni myDni = new Dni(1821828);
```


Usando miembros del package



- Fully qualified name

```
org.ieselcaminas.prog.util.Dni myDni;  
myDni = new org.ieselcaminas.prog.util.Dni(1821828);
```

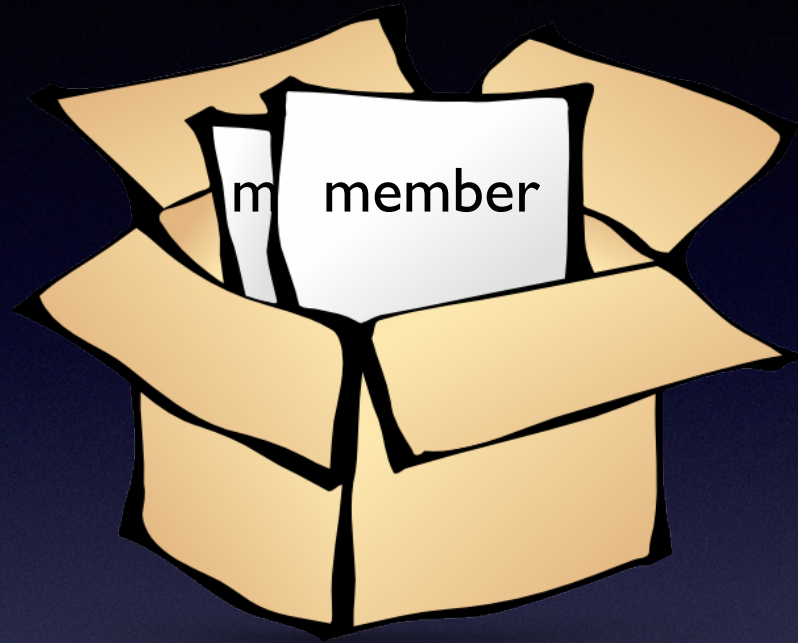
- Importando el miembro del package

```
import org.ieselcaminas.prog.util.Dni;  
Dni myDni = new Dni(1821828);
```

- Importando todo el package

```
import org.ieselcaminas.prog.util.*;  
Dni myDni = new Dni(1821828);
```


Usando miembros del package



- Fully qualified name

```
org.ieselcaminas.prog.util.Dni myDni;  
myDni = new org.ieselcaminas.prog.util.Dni(1821828);
```

- Importando el miembro del package

```
import org.ieselcaminas.prog.util.Dni;  
Dni myDni = new Dni(1821828);
```

- Importando todo el package

```
import org.ieselcaminas.prog.util.*;  
Dni myDni = new Dni(1821828);
```

Eliminar la ambigüedad

static import

```
package java.lang;
public class Math {
    public static final double PI = 3.141592653589793;
    public static double cos(double a)
    {
        ...
    }
}
```

```
double r = Math.cos(Math.PI * theta);
```


static import

```
package java.lang;
public class Math {
    public static final double PI = 3.141592653589793;
    public static double cos(double a)
    {
        ...
    }
}
```

```
import static java.lang.Math.PI;
double r = Math.cos(Math.PI * theta);
```


static import

```
package java.lang;
public class Math {
    public static final double PI = 3.141592653589793;
    public static double cos(double a)
    {
        ...
    }
}
```

```
import static java.lang.Math.PI;
double r = Math.cos(Math.PI * theta);
```

```
import static java.lang.Math.*;
double r = Math.cos(Math.PI * theta);
```


static import

```
package mypackage;  
public class MyConstants {  
    public static final double MAX_LENGTH = 6.1415;  
    public static final int NUM_VALUES = 6;  
}
```

```
import static mypackage.MyConstants.*;
```


Gestionando ficheros fuente y Class

- Llama al fichero fuente con el nombre de la clase
+ .java

Gestionando ficheros fuente y Class

- Llama al fichero fuente con el nombre de la clase + .java
- Coloca el fichero .java en una estructura de directorios de acuerdo a la estructura de paquetes:
ej: .../org/ieselcaminas/prog/util/Dni.java

Asignando a la variable **CLASSPATH**

Para mostrar el contenido de **CLASSPATH**:

```
En Windows:    C:\> set CLASSPATH  
En Unix:       % echo $CLASSPATH
```


Asignando a la variable **CLASSPATH**

Para mostrar el contenido de **CLASSPATH**:

```
En Windows:    C:\> set CLASSPATH
En Unix:       % echo $CLASSPATH
```

Para asignar contenido a **CLASSPATH**:

```
En Windows:    C:\> set CLASSPATH=C:\users\george\java\classes
En Unix:       % CLASSPATH=/home/george/java/classes
               % export CLASSPATH
```


Asignando a la variable **CLASSPATH**

Para mostrar el contenido de CLASSPATH:

```
En Windows:    C:\> set CLASSPATH
En Unix:       % echo $CLASSPATH
```

Para asignar contenido a CLASSPATH:

```
En Windows:    C:\> set CLASSPATH=C:\users\george\java\classes
En Unix:       % CLASSPATH=/home/george/java/classes
               % export CLASSPATH
```

Para borrar el contenido de CLASSPATH:

```
En Windows:    C:\> set CLASSPATH=
En Unix:       % unset CLASSPATH
               % export CLASSPATH
```


Devolución de un valor por un método

Un método vuelve al código que lo invocó cuando ...

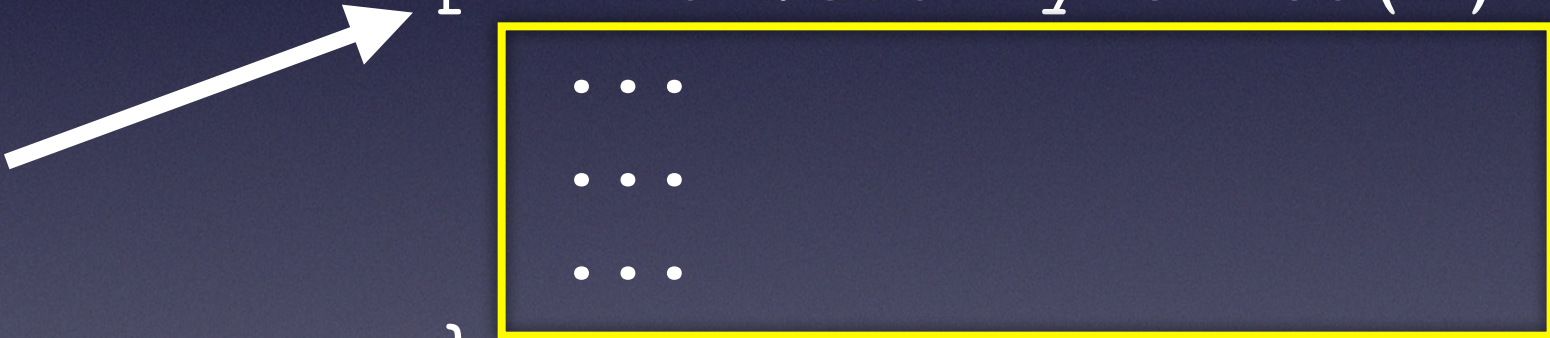
- Completa todas las instrucciones del método,
- Alcanzó una instrucción a return, o
- Lanza una excepción (estudiado en otro tema),

Devolución de un valor por un método

Un método vuelve al código que lo invocó cuando ...

- Completa todas las instrucciones del método,
- Alcanzó una instrucción a return, o
- Lanza una excepción (estudiado en otro tema),

```
...  
...  
...  
myMethod( )  
...  
...  
...
```

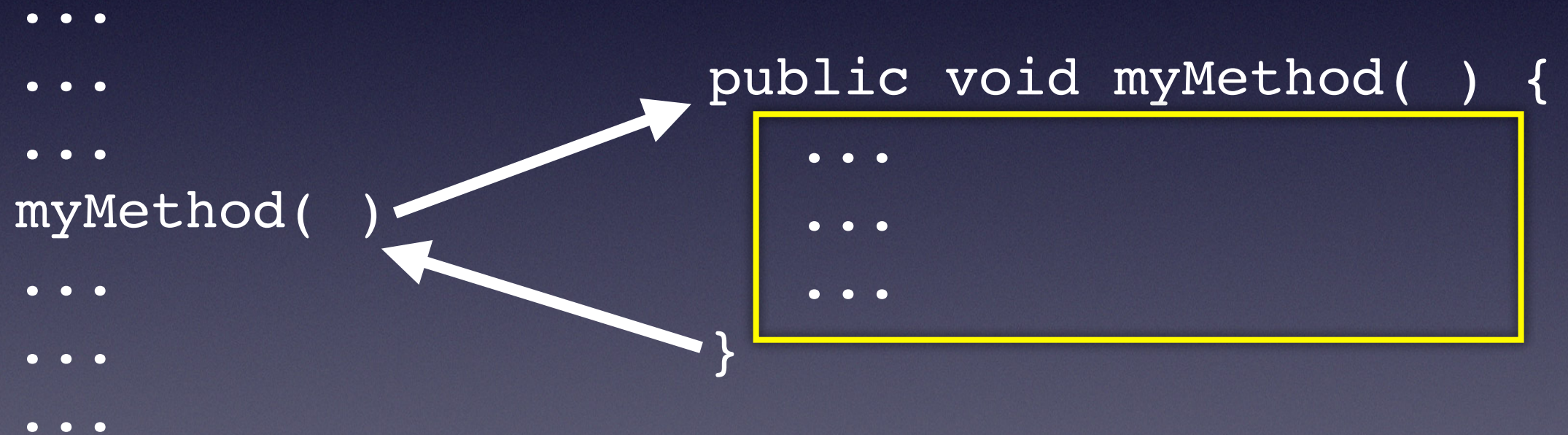


```
public void myMethod( ) {  
    ...  
    ...  
    ...  
}
```


Devolución de un valor por un método

Un método vuelve al código que lo invocó cuando ...

- Completa todas las instrucciones del método,
- Alcanzó una instrucción a return, o
- Lanza una excepción (estudiado en otro tema),



Devolución de un valor por un método

Un método vuelve al código que lo invocó cuando ...

- Completa todas las instrucciones del método,
- Alcanzó una instrucción a return, o
- Lanza una excepción (estudiado en otro tema),

```

...
...
...
myMethod( )
...
...
...

```

```

public void myMethod( ) {
    ...
    return;
    ...
}

```


Devolución de un valor por un método

Declaramos el tipo de valor que devuelve el método delante del nombre del método

```
public int getArea() {  
    return width * height;  
}
```


Devolución de un valor por un método

Declaramos el tipo de valor que devuelve el método delante del nombre del método

```
public int getArea() {  
    return width * height;  
}
```

```
public void setWidth(int newWidth) {  
    width = newWidth;  
    return;  
}
```



Cuando void, return es opcional

Controlando el acceso a una Clase



- Class:
- public
 - Sin modificador. package-private

Controlando el acceso a los miembros de una Clase



- Class:
- public
 - Sin modificador. package-private

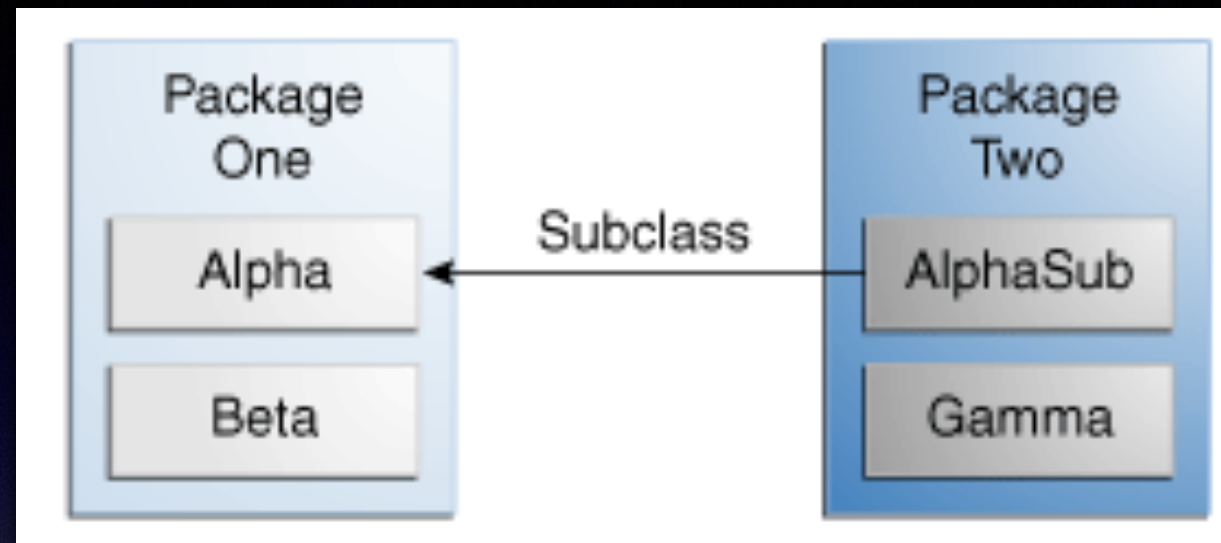
- Miembro:
(Atributo o método)
- public
 - protected
 - Sin modificador. package-private
 - private

Controlando el acceso a los miembros de una Clase



Modificador	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>Sin modificador</i>	Y	Y	N	N
private	Y	N	N	N

Controlando el acceso a los miembros de una Clase



Modifier member of Alpha	Alpha	Beta	AlphaSub	Gamma
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>no modifier</i>	Y	Y	N	N
private	Y	N	N	N

El modificador static

static = pertenece a la Clase, no a los objetos de esa Clase

```
public class Bicycle {  
  
    //instance variables  
    private int cadence;  
    private int gear;  
    private int speed;  
  
    // add a class variable for the  
    // number of Bicycle objects instantiated  
    private static int numberOfBicycles = 0;  
    ...  
}
```


El modificador static

static = pertenece a la Clase, no a los objetos de esa Clase

```
public Bicycle(int startCadence, int startSpeed,  
               int startGear){  
    gear = startGear;  
    cadence = startCadence;  
    speed = startSpeed;  
  
    ++numberOfBicycles;  
}
```


El modificador static

static = pertenece a la Clase, no a los objetos de esa Clase

```
public class Bicycle {  
    //instance variables  
    private int cadence;  
    private int gear;  
    private int speed;  
  
    // add a class variable for the  
    // number of Bicycle objects instantiated  
    private static int numberOfBicycles = 0;  
    ...  
}
```

```
    public static getNumBikes() {  
        return numberOfBicycles;  
    }
```


El modificador static

static = pertenece a la Clase, no a los objetos de esa Clase

```
public class Bicycle {  
    //instance variables  
    private int cadence;  
    private int gear;  
    private int speed;
```

```
    // add a class variable for the  
    // number of Bicycle objects instantiated  
    private static int numberOfBicycles = 0;  
    ...
```

```
}
```

```
    public static getNumBikes() {  
        return numberOfBicycles;  
    }
```

Los método estáticos solo pueden acceder a atributos estáticos

El modificador static

```
public class Bicycle {  
  
    //instance variables  
    private int cadence;  
    private int gear;  
    private int speed;  
  
    // add a class variable for the  
    // number of Bicycle objects instantiated  
    private static int numberOfBicycles = 0;  
    ...  
  
    public static int getNumBikes() {  
        return numberOfBicycles;  
    }  
}
```

Cómo invocar un método estático:

```
System.out.println("num. bikes = "+ Bicycle.getNumBikes());
```


Interfaces

Si su clase afirma implementar una interfaz, todos los métodos definidos por esa interfaz deben aparecer en su código fuente antes de que la clase se compile correctamente.

```
interface Talking {  
  
    // Headers of the methods that the  
    // interface force to implement go here  
    void talk();  
  
}
```

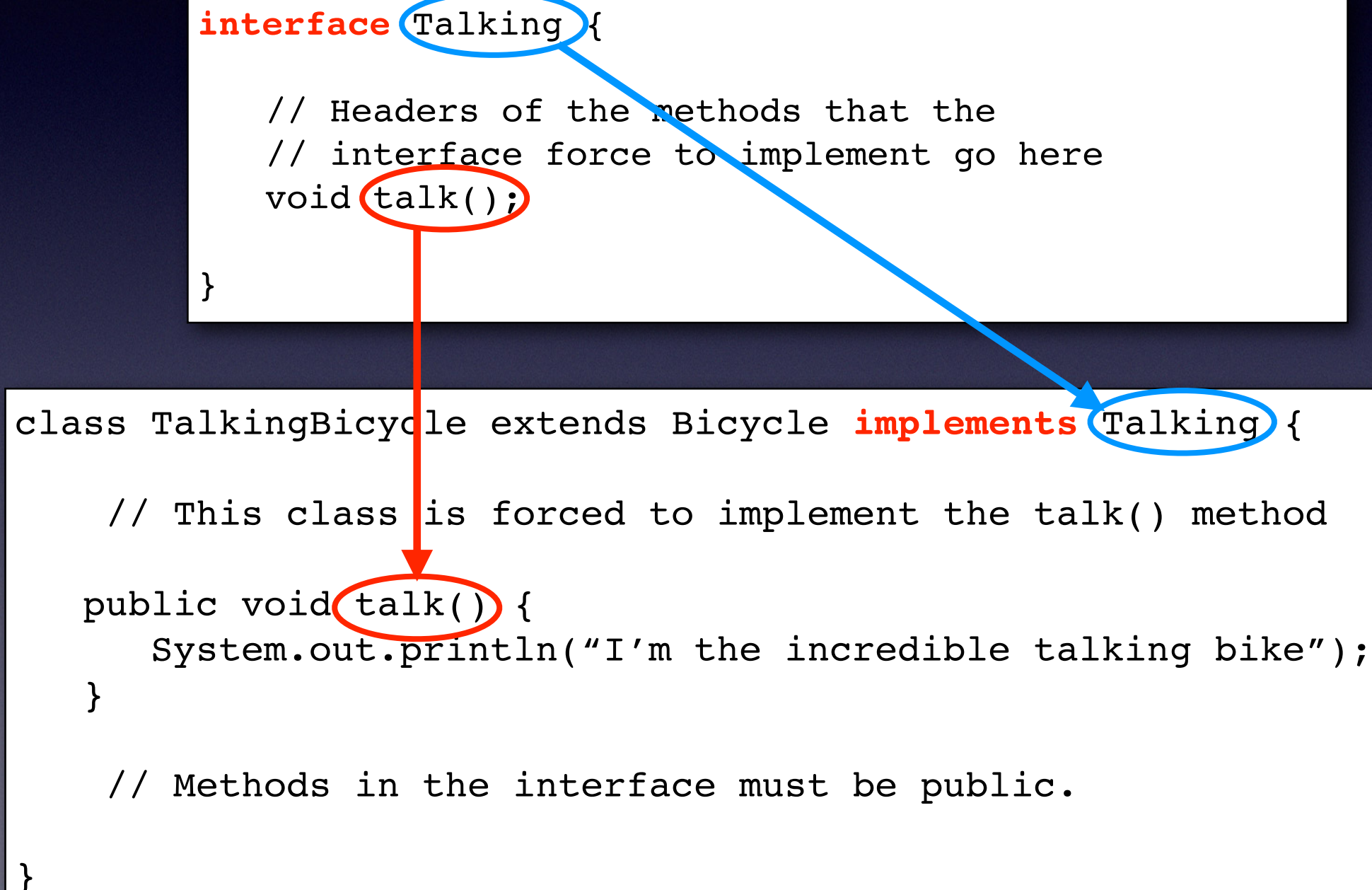
```
class TalkingBicycle extends Bicycle implements Talking {  
  
    // This class is forced to implement the talk() method  
  
    public void talk() {  
        System.out.println("I'm the incredible talking bike");  
    }  
  
    // Methods in the interface must be public.  
  
}
```


Interfaces

Si su clase afirma implementar una interfaz, todos los métodos definidos por esa interfaz deben aparecer en su código fuente antes de que la clase se compile correctamente.

```
interface Talking {  
  
    // Headers of the methods that the  
    // interface force to implement go here  
    void talk();  
  
}
```

```
class TalkingBicycle extends Bicycle implements Talking {  
  
    // This class is forced to implement the talk() method  
    public void talk() {  
        System.out.println("I'm the incredible talking bike");  
    }  
  
    // Methods in the interface must be public.  
  
}
```



Interfaces

```
interface Talking {  
    void talk();  
}
```

```
class Person implements Talking {  
    String dni;  
    String name;  
    public void talk() {  
        System.out.println("I am a person");  
    }  
}
```

```
class Bicycle implements Talking {  
    int cadence = 0;  
    int speed = 0;  
    int gear = 1;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
    public void talk() {  
        System.out.println("I'm the incredible talking bike");  
    }  
}
```


Interfaces

```
class UsingInterfaces {  
  
    public static void makeItTalk(Talking t) {  
        t.talk();  
    }  
  
    public static void main(String argv[]) {  
        Bicycle bike = new Bicycle();  
        Person person = new Person();  
        makeItTalk(bike);  
        makeItTalk(person);  
    }  
}
```


Ejercicios. Modifica tu proyecto Geometry:

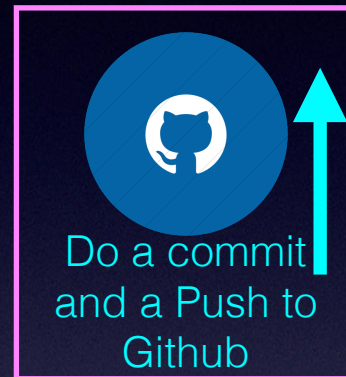
Diseña una clase llamada `Triangle`, que modele un triángulo con tres vértices. Se diseñará de la siguiente manera. La clase `MyTriangle` utiliza tres instancias de `Point` (creadas en un ejercicio anterior) como los tres vértices.

La clase contiene:

- Tres atributos `private: v1, v2, v3` (instancias de `Point`), para los tres vértices.
- Un constructor que genera un `Triangle` con 3 puntos, dando: `x1, y1, x2, y2, x3, y3`.
- An constructor sobrecargado que genere un `Triangle` dando 3 instancias de `Point`.
- Un método `toString()` que devuelve un string de la instancia en el formato "`Triangle @ (x1, y1), (x2, y2), (x3, y3)`".
- Un método `getPerimeter()` que devuelve la longitud del perímetro. Crea un método estático en la clase `Point` `distance()` que acepta dos points como parámetro y devuelve la distancia entre ellos.
- Un método `printType()`, que imprime "equilateral" si todos los lados son iguales, "isosceles" si dos son iguales, o "scalene" si los tres lados son diferentes.

Escribe la clase `Triangle`. También un programa de test (llamado `TestMyTriangle`) que compruebe todos los métodos de la clase.

Ejercicios. Modificar tu proyecto Geometry:



Sube la URL a la tarea individual de Aules

Ejercicio

Polynomials

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Crea un proyecto Polynomials

5. Diseña una clase llamada MyPolynomial, que modela polinomios de grado-n (ver ecuación), está diseñada como se muestra en el diagrama de clases.

La clase contiene:

- Una variable de instancia denominada coeffs, que almacena los coeficientes del polinomio de grado n en un array de double de tamaño n+1, donde c0 se mantiene en el índice 0.
- Un constructor MyPolynomial(coeffs:double...) Toma un número variable de doubles para inicializar el array coeffs, donde el primer argumento corresponde a C0. Los tres puntos se conocen como var args (número variable de argumentos), que es una nueva característica introducida en JDK 1.5. Acepta un array o una secuencia de argumentos separados por comas. El compilador empaqueta automáticamente los argumentos separados por comas en un array. Los tres puntos solo se pueden usar para el último argumento del método.

Ejercicio

Polynomials

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Crea un proyecto Polynomials

Pistas:

```
public class MyPolynomial {
    private double[] coeffs;
    public MyPolynomial(double... coeffs) { // varargs
        this.coeffs = coeffs; // varargs is treated as array
    }
    .....
}

// Test program
// Can invoke with a variable number of arguments
MyPolynomial p1 = new MyPolynomial(1.1, 2.2, 3.3);
MyPolynomial p1 = new MyPolynomial(1.1, 2.2, 3.3, 4.4, 5.5);
// Can also invoke with an array
double[] coeffs = {1.2, 3.4, 5.6, 7.8};
MyPolynomial p2 = new MyPolynomial(coeffs);
```


Polynomials

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Otro constructor que toma coeficientes de un archivo (del nombre de archivo dado), que tiene este formato:

```
Degree-n (int)
c0 (double)
c1 (double)
.....
.....
cn-1 (double)
cn (double)
(end-of-file)
```

Pistas:

```
public MyPolynomial(String filename) {
    Scanner in = null;
    try {
        in = new Scanner(new File(filename)); // open file
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    int degree = in.nextInt(); // read the degree
    coeffs = new double[degree+1]; // allocate the array
    for (int i=0; i<coeffs.length; ++i) {
        coeffs[i] = in.nextDouble();
    }
}
```

Para crear el archivo de texto para almacenar el polinomio tienes que seleccionar la carpeta raíz de tu proyecto en IntelliJ y luego ir al menú Archivo / nuevo / Archivo --> nombre: poly.txt (por ejemplo), y luego escribir el archivo con los coeficientes así, por ejemplo:

```
4
1.5
2.4
3.1
5.0
2.3
```

Donde 4 is el grado, y el polinomio es:

$2.3x^4 + 5.0x^3 + 3.1x^2 + 2.4x + 1.5$

Polynomials

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Un método `getDegree()` que devuelve el grado de este polinomio.

- Un método `toString()` que devuelve " $c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$ ".
- Un método `evaluate(double x)` que evalúan el polinomio para el x dado, sustituyendo la x en la expresión polinómica.
- Métodos `add()` y `multiply()` que suma y multiplica este polinomio con la instancia de `MyPolynomial` dada como parámetro (`another`) y devuelve una nueva instancia de `MyPolynomial` que contiene el resultado.

Escribe la clase `MyPolynomial`. También escribe un programa de prueba (llamado `TestMyPolynomial`) para probar todos los métodos definidos en la clase.

Pregunta: ¿Es necesario mantener el grado del polinomio como una variable de instancia en la clase `MyPolynomial` en Java?

Polynomials

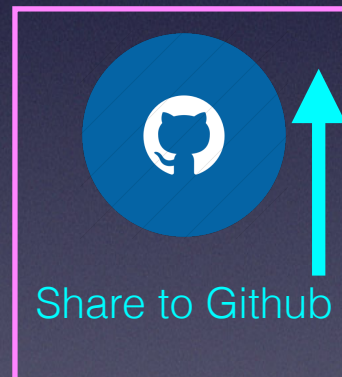
$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

Ejemplo multiplicación de dos polinomios:

$$\begin{array}{r} x^3 + 4x^2 - 2x + 6 \\ \quad \quad \quad 2x + 7 \\ \hline 7x^3 + 28x^2 - 14x + 42 \\ 2x^4 + 8x^3 - 4x^2 + 12x \\ \hline 2x^4 + 15x^3 + 24x^2 - 2x + 42 \end{array}$$

Suma:

$$\begin{array}{r} x^5 + 3x^3 + 4 \\ + 6x^6 + \quad + 4x^3 \\ \hline 6x^6 + x^5 + 7x^3 + 4 \end{array}$$



Sube la URL a la tarea individual
de Aules

Ejercicio.

En aules



Loterías del Estado

UN SORTEO 162



La Primitiva
JOKER

Este boleto sirve únicamente para la lectura de apuestas por un terminal en línea con un ordenador central

1	10	20	30	40	2	10	20	30	40	3	10	20	30	40	4	10	20	30	40	5	10	20	30	40	6	10	20	30	40	7	10	20	30	40	8	10	20	30	40
1	11	21	31	41	1	11	21	31	41	1	11	21	31	41	1	11	21	31	41	1	11	21	31	41	1	11	21	31	41	1	11	21	31	41	1	11	21	31	41
2	12	22	32	42	2	12	22	32	42	2	12	22	32	42	2	12	22	32	42	2	12	22	32	42	2	12	22	32	42	2	12	22	32	42	2	12	22	32	42
3	13	23	33	43	3	13	23	33	43	3	13	23	33	43	3	13	23	33	43	3	13	23	33	43	3	13	23	33	43	3	13	23	33	43	3	13	23	33	43
4	14	24	34	44	4	14	24	34	44	4	14	24	34	44	4	14	24	34	44	4	14	24	34	44	4	14	24	34	44	4	14	24	34	44	4	14	24	34	44
5	15	25	35	45	5	15	25	35	45	5	15	25	35	45	5	15	25	35	45	5	15	25	35	45	5	15	25	35	45	5	15	25	35	45	5	15	25	35	45
6	16	26	36	46	6	16	26	36	46	6	16	26	36	46	6	16	26	36	46	6	16	26	36	46	6	16	26	36	46	6	16	26	36	46	6	16	26	36	46
7	17	27	37	47	7	17	27	37	47	7	17	27	37	47	7	17	27	37	47	7	17	27	37	47	7	17	27	37	47	7	17	27	37	47	7	17	27	37	47
8	18	28	38	48	8	18	28	38	48	8	18	28	38	48	8	18	28	38	48	8	18	28	38	48	8	18	28	38	48	8	18	28	38	48	8	18	28	38	48
9	19	29	39	49	9	19	29	39	49	9	19	29	39	49	9	19	29	39	49	9	19	29	39	49	9	19	29	39	49	9	19	29	39	49	9	19	29	39	49

44

7

28

84

210

462

Si juega múltiple, marque aquí el número de apuestas

Si desea jugar al **JOKER** marque una X en la casilla → ☐ BIEN ☒ MAL ☒

PARTICIPA EN UN SORTEO:
JUEVES o SÁBADO

MARQUE CON X, EN AZUL O NEGRO, LOS NÚMEROS ELEGIDOS

Si desea jugar más veces las mismas apuestas no doble, arrugue ni rompa este boleto

Ejercicio.

Tic-Tac-Toe



Crea un programa para jugar a Tic-Tac-Toe (3 en raya)

Crearás una clase Main y una clase Board que almacenará el tablero, tendrá un método shoot(int row, int col, int player) donde jugador es 1 o 2. Crea otro método wins() que retorne si el jugador 1 gana, o el jugador 2 gana, o empata (empate) o el juego está en progreso.

Si eres lo suficientemente audaz, puedes crear una clase de Jugador para almacenar los nombres de los jugadores y usarlos en lugar de solo 1 y 2 como jugadores.

Ejercicio.

Tic-Tac-Toe



Para imprimir el tablero puedes hacer algo sencillo como esto:

```

  1  2  3
1
2    0
3
```

Victor enter row and col: